

CẨM NANG ÔN TẬP TOÀN DIỆN VÒNG LOẠI OLYMPIC AI 2026

Lý thuyết Cốt lõi, Đạo hàm Toán học, 50 Câu Đề Thi VOAİ Kèm Lời Giải & Ma Trận Tra Cứu Web

Biên soạn phục vụ Kỳ thi Tuyển chọn Đội tuyển Olympic Trí tuệ Nhân tạo HCMUS (Vòng loại 11/10/2026)

Tham chiếu Chuẩn kiến thức: Olympic Tin học & AI Việt Nam (VOAI 2026), International Olympiad in AI (IOAI 2026) & OLP AI 2025

Abstract

Cẩm nang học thuật này được thiết kế theo cấu trúc chuyên khảo toàn diện chuẩn bài báo khoa học (Academic Reference Monograph), nhằm cung cấp toàn bộ hệ thống lý thuyết, công thức toán học, **giải thích chi tiết từng biến số, tham số, chiều dữ liệu và trực giác cơ chế** cho thí sinh tham dự Vòng loại Olympic AI HCMUS 2026. Nội dung bao quát 5 chương trọng tâm: **Machine Learning Cổ điển**, **Deep Learning & PyTorch Framework**, **Thị giác Máy tính (CV)**, **Xử lý Ngôn ngữ Tự nhiên (NLP)** và **Toán & Xác suất Thống kê cho AI**. Toàn bộ các công thức toán học, bài toán tính tay kinh điển (Ma trận trực giao, Hessian, Conv2D dimensions, Bayes base-rate fallacy, Entropy, Information Gain, IoU, F1-score, Cosine similarity) và 24 bài đề thi trắc nghiệm được đối chiếu minh bạch. Đặc biệt, tài liệu bổ sung **Trọn bộ 50 câu trắc nghiệm thực chiến đề thi thử VOAİ 2026 kèm đáp án và lời giải chi tiết 100% từng câu**, cùng **Bảng Ma Trận Tra Cứu Nhanh kết nối trực tiếp với Web App Ôn Thi** (<https://lenamkhanhh.github.io/OLPAİ26-Khanhdz/>) để thí sinh luyện đề và tra cứu cơ sở đáp án tức thì.

Contents

1	Machine Learning Cổ Điển (Classical Machine Learning)	2
1.1	Thuật toán k-NN (k-Nearest Neighbors) & Bản chất Lazy Learner	2
1.2	Support Vector Machines (SVM) & Kernel Trick	3
1.3	Cây Quyết Định (Decision Tree), Entropy & Information Gain	4
1.4	Random Forest & Phương Pháp Ensemble Learning	5
1.5	Bias-Variance Tradeoff & Kỹ thuật Cross-Validation	5
1.6	Regularization: L2 (Ridge) vs L1 (Lasso) vs ElasticNet	6
1.7	Các Độ Đo Đánh Giá Phân Lớp (Classification Metrics)	6
1.8	K-Means Clustering & Silhouette Analysis	7
1.9	Kỹ thuật Xử lý Mất Cân Bằng Lớp (Imbalanced Data)	8
2	Deep Learning Cơ Bản & PyTorch Framework	8
2.1	Perceptron, MLP & Định Lý Xấp Xi Phô Quát	8
2.2	Phân Tích Chi Tiết Các Hàm Kích Hoạt (Activation Functions)	8
2.3	Quy Trình Huấn Luyện Chuẩn PyTorch (Training Loop)	9
2.4	Các Hàm Mất Mát (Loss Functions) & Bẫy Double-Softmax	9
2.5	Thuật Toán Tối Ưu Hóa: SGD, Momentum, RMSProp & Adam	10
2.6	Khởi Tạo Trọng Số (Weight Initialization)	10
2.7	Batch Normalization vs Layer Normalization	11
2.8	Cơ Chế Inverted Dropout	11
3	Thị Giác Máy Tính (Computer Vision)	11
3.1	Công Thức Kích Thước Không Gian Tầng Conv2D	11
3.2	Pooling, Global Average Pooling (GAP) & Receptive Field	12

3.3	Kiến Trúc CNN Kinh Điển & Đột Phá ResNet	12
3.4	Phân Biệt Skip Connection: ResNet (ADD) vs U-Net (CONCAT)	13
3.5	Object Detection: IoU, NMS, mAP, YOLO vs Faster R-CNN	13
4	Xử Lý Ngôn Ngữ Tự Nhiên (Natural Language Processing)	13
4.1	Pipeline Tiền Xử Lý Văn Bản Chuẩn 5 Bước	13
4.2	Các Phương Pháp Biểu Diễn Từ (Word Representations)	14
4.3	Độ Đo Cosine Similarity	14
4.4	Mạng Hồi Quy Tuần Tự: RNN, LSTM & GRU	14
4.4.1	Mạng No-ron Hồi Quy Cổ Điển (Vanilla RNN)	14
4.4.2	Mạng LSTM (Long Short-Term Memory — Hochreiter & Schmidhuber, 1997)	14
4.4.3	Mạng GRU (Gated Recurrent Unit — Cho et al., 2014)	15
4.5	Kiến Trúc Transformer (Vaswani et al., 2017)	15
4.6	Các Độ Đo Đánh Giá NLP: BLEU, SacreBLEU, ROUGE & Perplexity	16
5	Toán & Xác Suất Thống Kê Cho Trí Tuệ Nhân Tạo	17
5.1	Định Lý Bayes & Nguy Biện Tỷ Lệ Cơ Sở (Base-Rate Fallacy)	17
5.2	Tính Chất của Kỳ Vọng, Phương Sai & Ma Trận Tương Quan	17
5.3	Ước Lượng Tham Số: MLE vs MAP & Mối Liên Hệ Với Regularization	17
5.4	Kiểm Định Giả Thuyết Thống Kê & Ý Nghĩa Của p-value	18
5.5	Đại Số Tuyến Tính Nâng Cao Cho AI: Ma Trận Trục Giao & Ma Trận Hessian	18
5.6	Kỹ Thuật Cross-Validation Cho Time-Series & Chống Rò Rỉ Dữ Liệu (Data Leakage)	19
6	Bảng 24 Bẫy Đề Thi Kinh Điển (Lỗi Hay Mắc: SAI → ĐÚNG)	19
7	Chuyên Đề Tác Vụ Thực Chiến OLP AI 2025 & 2026 (Khung 5 Bước)	20
7.1	Tác vụ 1 (Đề OLP AI 2025): Nhận Diện Ngôn Ngữ Ký Hiệu từ Video	20
7.2	Tác vụ 2 (Đề OLP AI 2025): Dịch Máy Thương Mại Điện Tử Hoa — Việt	21
7.3	Tác vụ 3 (Đề xuất OLP 2026): Phát Hiện Bệnh Lá Cây Trồng Trên Thiết Bị Di Động	21
7.4	Tác vụ 4 (Đề xuất OLP 2026): Dự Đoán Sinh Viên Bỏ Học & Explainable AI (XAI)	21
8	Bộ Đề Luyện Thi Thực Chiến Chuẩn VOAI 2026 (50 Câu) — Kèm Đáp Án & Giải Thích Chi Tiết	23
9	Ma Trận Đối Chiếu Tra Cứu Web App & Lộ Trình Ôn Luyện Nước Rút	44
9.1	Phương Pháp Kết Hợp Web App Thi Thử & Handbook PDF	44
9.2	Bảng Ma Trận Ánh Xạ Chủ Đề Giữa Web App Và Handbook PDF	44
9.3	Rubric Tự Chấm Điểm 4 Bài Tự Luận OLP AI (10 Điểm/Câu)	44

§1 Machine Learning Cổ Điển (Classical Machine Learning)

§1.1 Thuật toán k-NN (k-Nearest Neighbors) & Bản chất Lazy Learner

Định nghĩa hình thức: k-NN là thuật toán phi tham số (non-parametric), thuộc lớp *Instance-based Learning* hay *Lazy Learner*. Thuật toán **không có pha huấn luyện tham số** (không tối ưu trọng số $\mathbf{w}$, b). Tại pha suy luận (Inference), với một truy vấn $\mathbf{x}_q \in \mathbb{R}^d$, mô hình tính khoảng cách từ $\mathbf{x}_q$ tới toàn bộ N điểm trong tập huấn luyện $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, xác

định tập k láng giềng gần nhất $\mathcal{N}_k(\mathbf{x}_q)$, và thực hiện biểu quyết:

$$\hat{y} = \arg \max_{c \in \mathcal{C}} \sum_{i \in \mathcal{N}_k(\mathbf{x}_q)} \mathbb{I}(y_i = c) \quad \text{hoặc} \quad \hat{y} = \arg \max_{c \in \mathcal{C}} \sum_{i \in \mathcal{N}_k(\mathbf{x}_q)} \frac{1}{D(\mathbf{x}_q, \mathbf{x}_i) + \epsilon} \mathbb{I}(y_i = c) \quad (1)$$

Giải thích chi tiết các ký hiệu và tham số trong công thức:

- $\mathbf{x}_q \in \mathbb{R}^d$: Điểm dữ liệu truy vấn cần dự đoán (Query vector) với d đặc trưng số thực.
- $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$: Tập dữ liệu huấn luyện gồm N mẫu đã gán nhãn.
- $y_i \in \mathcal{C}$: Nhãn lớp thực tế của mẫu huấn luyện thứ i . $\mathcal{C} = \{1, 2, \dots, C\}$ là tập hợp các lớp phân loại.
- $\mathcal{N}_k(\mathbf{x}_q)$: Tập hợp chỉ số của đúng k điểm trong tập huấn luyện có khoảng cách $D(\mathbf{x}_q, \mathbf{x}_i)$ nhỏ nhất tới $\mathbf{x}_q$.
- $\mathbb{I}(\cdot)$: Hàm chỉ thị (Indicator function), nhận giá trị 1 nếu mệnh đề bên trong đúng ($y_i = c$), và 0 nếu sai.
- $D(\mathbf{x}_q, \mathbf{x}_i)$: Khoảng cách toán học giữa hai điểm dữ liệu.
- $\epsilon > 0$: Số thực dương cực nhỏ (ví dụ $\epsilon = 10^{-6}$) nhằm triệt tiêu nguy cơ chia cho 0 khi mẫu truy vấn trùng khít với mẫu train.
- $\hat{y}$: Nhãn lớp đầu ra được dự đoán cho $\mathbf{x}_q$.

Không gian khoảng cách (Minkowski Metric L_p):

$$D_p(\mathbf{x}, \mathbf{z}) = \left(\sum_{j=1}^d |x_j - z_j|^p \right)^{1/p} \quad (2)$$

- x_j, z_j : Giá trị tại chiều đặc trưng thứ j của hai vector $\mathbf{x}$ và $\mathbf{z}$.
- $p = 1$: Khoảng cách **Manhattan** (L_1), tổng khoảng cách dịch chuyển dọc theo các trục tọa độ.
- $p = 2$: Khoảng cách **Euclid** (L_2), đường thẳng nối ngắn nhất trong không gian Euclid.
- $p \rightarrow \infty$: Khoảng cách **Chebyshev** (L_∞), lấy giá trị chênh lệch lớn nhất trên từng chiều đơn lẻ: $\max_j |x_j - z_j|$.

[BẢY THI CẦN TRÁNH] — k-NN không có Training Phase & Bắt buộc Feature Scaling

- **Độ phức tạp tính toán:** Pha huấn luyện tốn $\mathcal{O}(1)$ thời gian (chỉ nạp dữ liệu vào RAM). Pha dự đoán tốn $\mathcal{O}(N \cdot d)$ phép toán vì phải duyệt qua toàn bộ N điểm trong cơ sở dữ liệu. Do đó khi N lên tới hàng triệu mẫu, k-NN cực kỳ chậm khi inference nếu không có cấu trúc chỉ mục cây không gian (KD-Tree, Ball-Tree).
- **Bắt buộc chuẩn hóa đặc trưng (Feature Scaling):** Nếu thuộc tính thu nhập $x_1 \in [0, 100\,000\,000]$ và độ tuổi $x_2 \in [18, 65]$, khoảng cách Euclid sẽ bị áp đảo hoàn toàn bởi x_1 , triệt tiêu sức ảnh hưởng của x_2 . Do đó bắt buộc phải chuẩn hóa thang đo (StandardScaler hoặc MinMaxScaler).
- **Ảnh hưởng của siêu tham số k :**
 - $k = 1$: Ranh giới quyết định (Voronoi tessellation) ôm chặt từng điểm nhiều $\implies$ **Overfitting** (High Variance).
 - k lớn tiến tới N : Ranh giới phẳng lì, dự đoán nghiêng về lớp chiếm đa số $\implies$ **Underfitting** (High Bias).

§1.2 Support Vector Machines (SVM) & Kernel Trick

Mô hình hình học: Xét bài toán phân lớp nhị phân với siêu phẳng $\mathbf{w}^T \mathbf{x} + b = 0$. Khoảng cách hình học từ điểm bất kỳ đến siêu phẳng là $\gamma = \frac{|\mathbf{w}^T \mathbf{x} + b|}{\|\mathbf{w}\|_2}$. Để tối đa hóa lề hình học phân tách giữa hai lớp $\gamma_{\text{margin}} = \frac{2}{\|\mathbf{w}\|_2}$, bài toán quy về tối thiểu hóa $\|\mathbf{w}\|_2^2$.

Bài toán Soft-Margin Primal: Cho phép các điểm vi phạm lề thông qua biến bù $\xi_i \geq 0$:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_{i=1}^N \xi_i \quad \text{thỏa mãn} \quad y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad \forall i \quad (3)$$

Giải thích chi tiết các thành phần:

- $\mathbf{w} \in \mathbb{R}^d$: Vector trọng số pháp tuyến vuông góc với siêu phẳng phân cách. Tối thiểu hóa $\frac{1}{2}\|\mathbf{w}\|_2^2$ tương đương với tối đa hóa độ rộng lề $\frac{2}{\|\mathbf{w}\|_2}$.
- $b \in \mathbb{R}$: Hệ số chệch (bias / intercept), xác định độ dịch chuyển vị trí của siêu phẳng so với gốc tọa độ.
- $\mathbf{x}_i \in \mathbb{R}^d, y_i \in \{-1, +1\}$: Vector đặc trưng và nhãn nhị phân của mẫu huấn luyện thứ i .
- $\xi_i \geq 0$ (Slack Variable — Biến bù vi phạm): Đo lường khoảng cách mà mẫu thứ i vi phạm vùng an toàn:
 - $\xi_i = 0$: Mẫu nằm hoàn toàn ngoài lề (phân loại đúng, chuẩn xác tuyệt đối).
 - $0 < \xi_i \leq 1$: Mẫu nằm lọt vào trong vùng lề an toàn nhưng vẫn ở đúng phía siêu phẳng (phân loại đúng nhưng lấn lề).
 - $\xi_i > 1$: Mẫu nằm sai phía siêu phẳng (mô hình phân loại sai).
- $C > 0$: Siêu tham số điều hòa (Regularization penalty). Kiểm soát sự đánh đổi giữa độ rộng lề và sai số:
 - $C \rightarrow \infty$ (Hard-Margin): Phạt cực nặng mọi vi phạm $\Rightarrow$ lề hẹp $\Rightarrow$ Dễ **Overfitting**.
 - C nhỏ: Chấp nhận bỏ qua nhiều điểm vi phạm để mở rộng lề $\Rightarrow$ lề rộng $\Rightarrow$ Dễ **Underfitting**.

Dạng đối ngẫu (Dual Formulation) & Kernel Trick:

$$\max_{\alpha} \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j) \quad \text{thỏa mãn} \quad 0 \leq \alpha_i \leq C, \sum_{i=1}^N \alpha_i y_i = 0 \quad (4)$$

- $\alpha_i \geq 0$: Nhân tử Lagrange (Lagrange Multiplier) ứng với ràng buộc của mẫu thứ i .
- **Support Vectors (Vector hỗ trợ)**: Là các mẫu có $\alpha_i > 0$. Siêu phẳng tối ưu **chỉ phụ thuộc duy nhất vào các Support Vectors này**: $\mathbf{w} = \sum_{i \in \text{SV}} \alpha_i y_i \mathbf{x}_i$. Mọi điểm có $\alpha_i = 0$ nếu bị xóa khỏi tập train thì siêu phẳng hoàn toàn không thay đổi!
- $K(\mathbf{x}_i, \mathbf{x}_j) = \langle \Phi(\mathbf{x}_i), \Phi(\mathbf{x}_j) \rangle$: Hàm nhân (Kernel function), cho phép tính tích vô hướng trong không gian chiếu đặc trưng nhiều chiều/vô hạn chiều mà không cần ánh xạ tường minh Φ .

Hàm nhân RBF (Radial Basis Function / Gaussian Kernel):

$$K(\mathbf{x}, \mathbf{z}) = \exp(-\gamma \|\mathbf{x} - \mathbf{z}\|_2^2) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{z}\|_2^2}{2\sigma^2}\right), \quad \gamma = \frac{1}{2\sigma^2} \quad (5)$$

- σ : Độ lệch chuẩn (bán kính chuông Gaussian).
- γ : Tham số điều chỉnh bán kính ảnh hưởng của từng điểm dữ liệu:
 - γ rất lớn: σ rất hẹp, mỗi điểm dữ liệu chỉ ảnh hưởng cục bộ ngay sát nó $\Rightarrow$ biên quyết định uốn lượn ôm sát từng điểm dữ liệu $\Rightarrow$ **Overfitting nghiêm trọng**.
 - γ nhỏ: σ rất rộng, ảnh hưởng lan tỏa phẳng mướt $\Rightarrow$ biên quyết định thẳng đều $\Rightarrow$ **Underfitting**.

§1.3 Cây Quyết Định (Decision Tree), Entropy & Information Gain

Đo lường độ hỗn loạn (Shannon Entropy): Cho tập dữ liệu S gồm C lớp với xác suất xuất hiện lớp i là p_i :

$$H(S) = - \sum_{i=1}^C p_i \log_2(p_i) \quad (\text{đơn vị: bit}) \quad (6)$$

- S : Tập hợp dữ liệu tại nút đang xét.
- C : Tổng số lớp mục tiêu.
- $p_i = \frac{|S_i|}{|S|}$: Tỷ lệ (xác suất) các mẫu thuộc lớp i trong tập S .
- Cơ sở $\log_2$: Đo lường lượng thông tin theo đơn vị **bit**. Nếu $p_i = 0$, quy ước $p_i \log_2(p_i) = 0$.
- $H(S) = 0$ khi nút thuần khiết hoàn toàn (tất cả các mẫu thuộc về đúng 1 lớp).

- $H(S) = \log_2(C)$ khi các lớp phân bố đều hoàn toàn (độ hỗn loạn đạt cực đại).

Chỉ số độ tinh khiết Gini (Gini Impurity — Dùng trong thuật toán CART):

$$\text{Gini}(S) = 1 - \sum_{i=1}^C p_i^2 \quad (7)$$

Độ lợi thông tin (Information Gain — Dùng trong thuật toán ID3):

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v) \quad (8)$$

- A : Thuộc tính (Feature) được đưa ra để phân nhánh.
- $\text{Values}(A)$: Tập hợp tất cả các giá trị rời rạc có thể có của thuộc tính A .
- S_v : Tập con các mẫu trong S có giá trị thuộc tính A bằng v , tức $S_v = \{\mathbf{x} \in S \mid A(\mathbf{x}) = v\}$.
- $|S|, |S_v|$: Lực lượng (số lượng phần tử) của tập S và tập con S_v .
- Thuật toán chọn phân nhánh tại thuộc tính A có $IG(S, A)$ lớn nhất (làm giảm độ hỗn loạn nhiều nhất).

[VÍ DỤ TÍNH TOÁN TAY] — Tính toán Entropy & Information Gain từng bước

Cho node cha gồm 12 mẫu: 6 mẫu Đỏ ($p_1 = 0.5$) và 6 mẫu Xanh ($p_2 = 0.5$).

- Entropy node cha: $H(\text{Cha}) = -(0.5 \log_2 0.5 + 0.5 \log_2 0.5) = -(-0.5 - 0.5) = 1.0$ bit.
- Giả sử thuộc tính A chia tập thành 2 node con:
 - Nhánh trái (S_1 , 6 mẫu): 5 Đỏ, 1 Xanh $\implies p_1 = \frac{5}{6}, p_2 = \frac{1}{6}$. $H(S_1) = -(\frac{5}{6} \log_2 \frac{5}{6} + \frac{1}{6} \log_2 \frac{1}{6}) \approx 0.650$ bit.
 - Nhánh phải (S_2 , 6 mẫu): 1 Đỏ, 5 Xanh $\implies p_1 = \frac{1}{6}, p_2 = \frac{5}{6}$. $H(S_2) = -(\frac{1}{6} \log_2 \frac{1}{6} + \frac{5}{6} \log_2 \frac{5}{6}) \approx 0.650$ bit.
- Entropy kỳ vọng sau phân nhánh: $H(S, A) = \frac{6}{12} H(S_1) + \frac{6}{12} H(S_2) = 0.5(0.650) + 0.5(0.650) = 0.650$ bit.
- Độ lợi thông tin: $IG(S, A) = H(\text{Cha}) - H(S, A) = 1.0 - 0.650 = 0.350$ bit.

§1.4 Random Forest & Phương Pháp Ensemble Learning

- **Bagging (Bootstrap Aggregating)**: Rút mẫu ngẫu nhiên có hoàn lại (Bootstrap) từ N mẫu ban đầu để tạo ra B tập huấn luyện khác nhau cho B cây con.
- **Bản chất toán học của Out-of-Bag (OOB)**: Xác suất một mẫu bất kỳ KHÔNG được rút trúng trong 1 lần chọn là $1 - \frac{1}{N}$. Khi rút N lần độc lập, xác suất mẫu đó hoàn toàn không rơi vào tập train của một cây là:

$$\lim_{N \rightarrow \infty} \left(1 - \frac{1}{N}\right)^N = e^{-1} \approx 0.3679 \approx 36.8\% \quad (9)$$

$\implies$ Trung bình 36.8% dữ liệu không được huấn luyện trên cây con đó (gọi là mẫu OOB), dùng làm tập kiểm thử nội bộ tự nhiên để tính **OOB Error** mà không cần tốn riêng tập validation.

- **Feature Randomness**: Tại mỗi node của cây quyết định, chỉ chọn ngẫu nhiên m thuộc tính trong tổng số d thuộc tính để tìm điểm cắt tối ưu ($m = \lfloor \sqrt{d} \rfloor$ cho bài toán phân loại; $m = \lfloor d/3 \rfloor$ cho hồi quy). Điều này làm triệt tiêu sự tương quan giữa các cây, giúp giảm mạnh **Variance**.

§1.5 Bias-Variance Tradeoff & Kỹ thuật Cross-Validation

Phân rã sai số kỳ vọng (Expected Prediction Error):

$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{\left(f(x) - \mathbb{E}[\hat{f}(x)]\right)^2}_{\text{Bias}^2 \text{ (Độ chệch)}} + \underbrace{\mathbb{E}\left[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2\right]}_{\text{Variance (Phương sai)}} + \underbrace{\sigma_\epsilon^2}_{\text{Nhiều không thể khử}} \quad (10)$$

- $f(x)$: Hàm chân lý thực tế sinh dữ liệu: $y = f(x) + \epsilon$.
- $\hat{f}(x)$: Hàm dự đoán do mô hình học được trên một tập dữ liệu ngẫu nhiên.
- $\mathbb{E}[\hat{f}(x)]$: Giá trị dự đoán trung bình của mô hình khi huấn luyện qua vô số tập dữ liệu độc lập.
- Bias²: Đo lường sai khác giữa kỳ vọng của mô hình và chân lý khách quan. Bias cao $\implies$ mô hình quá cứng nhắc, bỏ sót quy luật $\implies$ **Underfitting**.
- Variance: Đo lường độ biến động của dự đoán mô hình khi tập huấn luyện thay đổi. Variance cao $\implies$ mô hình quá nhạy cảm với từng điểm dữ liệu ngẫu nhiên $\implies$ **Overfitting**.
- σ_ϵ^2 : Nhiễu ngẫu nhiên thực tế (Irreducible Error) không thể triệt tiêu.
- **Stratified K-Fold**: Bắt buộc sử dụng cho bài toán lệch lớp (Imbalanced Data) để đảm bảo tỉ lệ phân bố giữa các nhãn ở mỗi Fold giống hệt phân phối trên toàn bộ tập dữ liệu gốc.

§1.6 Regularization: L2 (Ridge) vs L1 (Lasso) vs ElasticNet

$$\text{Ridge } (L_2) : \quad \mathcal{L}_{\text{Ridge}}(\mathbf{w}) = \mathcal{L}_0(\mathbf{w}) + \frac{\lambda}{2} \|\mathbf{w}\|_2^2 = \mathcal{L}_0(\mathbf{w}) + \frac{\lambda}{2} \sum_{j=1}^d w_j^2 \quad (11)$$

$$\text{Lasso } (L_1) : \quad \mathcal{L}_{\text{Lasso}}(\mathbf{w}) = \mathcal{L}_0(\mathbf{w}) + \lambda \|\mathbf{w}\|_1 = \mathcal{L}_0(\mathbf{w}) + \lambda \sum_{j=1}^d |w_j| \quad (12)$$

$$\text{ElasticNet} : \quad \mathcal{L}_{\text{EN}}(\mathbf{w}) = \mathcal{L}_0(\mathbf{w}) + r\lambda \|\mathbf{w}\|_1 + \frac{1-r}{2} \lambda \|\mathbf{w}\|_2^2 \quad (13)$$

Giải thích chi tiết các thành phần:

- $\mathcal{L}_0(\mathbf{w})$: Hàm mất mát nguyên bản trên tập dữ liệu huấn luyện (MSE trong hồi quy tuyến tính hoặc Cross-Entropy trong Logistic Regression).
- $\mathbf{w} = [w_1, w_2, \dots, w_d]^T \in \mathbb{R}^d$: Vector trọng số của mô hình.
- $\lambda > 0$: Hệ số điều hòa (Regularization Strength). λ càng lớn, hàm phạt càng ép các trọng số phải co nhỏ lại để chống Overfitting.
- $r \in [0, 1]$: Tỷ lệ hòa trộn giữa chuẩn L1 và L2 trong ElasticNet.
- **Cơ chế hình học của Lasso (L_1)**: Vùng ràng buộc của chuẩn L_1 là một hình khối đa diện (Polyhedron) có các góc nhọn nằm chính xác trên các trục tọa độ. Khi đường mức của hàm mất mát tiếp xúc với khối đa diện, điểm tiếp xúc tối ưu có xác suất cực cao rơi đúng vào các đỉnh góc nhọn này. Tại đó, một loạt các tọa độ $w_j = 0$ chính xác tuyệt đối $\implies$ **Lasso thực hiện Lựa chọn đặc trưng tự động (Feature Selection), tạo tính thưa (Sparsity)**.
- **Cơ chế hình học của Ridge (L_2)**: Vùng ràng buộc là hình cầu tròn nhẵn, ép các trọng số co nhỏ dần về tiệm cận 0 nhưng **không bao giờ triệt tiêu hoàn toàn bằng 0**. Công thức nghiệm đóng của Ridge: $\mathbf{w}_{\text{Ridge}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$, số hạng $\lambda \mathbf{I}$ đảm bảo ma trận luôn khả nghịch ngay cả khi các cột bị đa cộng tuyến (Multicollinearity).

§1.7 Các Độ Đo Đánh Giá Phân Lớp (Classification Metrics)

Bảng Ma trận nhầm lẫn (Confusion Matrix):

- TP (True Positive): Mẫu thực tế Dương, mô hình đoán Dương (Đúng).
- FP (False Positive): Mẫu thực tế Âm, mô hình đoán Dương (Báo động giả / Sai lầm Loại I).
- FN (False Negative): Mẫu thực tế Dương, mô hình đoán Âm (Bỏ sót tội phạm / Sai lầm Loại II).
- TN (True Negative): Mẫu thực tế Âm, mô hình đoán Âm (Đúng).

Công thức các chỉ số cơ bản:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN} \quad (14)$$

$$F_1\text{-score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN} \quad (15)$$

$$F_\beta\text{-score} = (1 + \beta^2) \cdot \frac{\text{Precision} \cdot \text{Recall}}{\beta^2 \cdot \text{Precision} + \text{Recall}} \quad (16)$$

- $\beta = 1$: F_1 -score coi trọng Precision và Recall ngang nhau.
- $\beta = 2$: F_2 -score coi trọng **Recall gấp đôi Precision** (ưu tiên tối đa việc không bỏ sót mẫu dương: phát hiện bệnh hiểm nghèo, an ninh).
- $\beta = 0.5$: $F_{0.5}$ -score coi trọng **Precision gấp đôi Recall** (ưu tiên độ chuẩn xác khi đưa ra quyết định: lọc spam mail, cấp tín dụng).

Chiến lược tổng hợp F1 trong bài toán đa lớp (C lớp):

- **Macro F1**: $\text{Macro-F1} = \frac{1}{C} \sum_{c=1}^C F_{1,c}$. Tính trung bình không trọng số của F1 từng lớp. Coi mọi lớp có vai trò ngang nhau $\implies$ **Rất nhạy cảm với lớp thiểu số**.
- **Micro F1**: Tính tổng toàn cục $\sum TP, \sum FP, \sum FN$ rồi áp dụng công thức F1. Bằng chính xác Accuracy trên bài toán phân loại đơn nhãn.
- **Weighted F1**: Tính trung bình có trọng số theo số lượng mẫu thực tế của từng lớp: $\sum_{c=1}^C \frac{N_c}{N} F_{1,c}$.
- **Đường cong PR (Precision-Recall Curve)**: Khi dữ liệu bị mất cân bằng trầm trọng (ví dụ gian lận thẻ tín dụng 0.1%), đường cong ROC-AUC sẽ bị lạc quan giả tạo do TN quá lớn. Bắt buộc phải dùng **PR-AUC** để đánh giá chính xác năng lực mô hình.

§1.8 K-Means Clustering & Silhouette Analysis**Hàm mục tiêu Inertia (Within-Cluster Sum of Squares):**

$$J = \sum_{k=1}^K \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|_2^2 \quad (17)$$

- K : Số lượng cụm cần phân chia.
- C_k : Tập hợp các điểm dữ liệu được gán vào cụm thứ k .
- $\boldsymbol{\mu}_k = \frac{1}{|C_k|} \sum_{\mathbf{x} \in C_k} \mathbf{x}$: Tọa độ trọng tâm (Centroid) của cụm thứ k .
- **Thuật toán K-Means++**: Khởi tạo trọng tâm đầu tiên ngẫu nhiên; các trọng tâm tiếp theo được chọn với xác suất tỷ lệ thuận với bình phương khoảng cách tới trọng tâm gần nhất đã chọn ($P(x) \propto D(x)^2$). Giúp tránh tối thiểu cục bộ tồi tệ.

Hệ số Silhouette đánh giá độ gắn kết của cụm:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \in [-1, 1] \quad (18)$$

- $a(i)$: Khoảng cách trung bình từ điểm i tới tất cả các điểm khác trong cùng cụm (độ kết tụ nội cụm, muốn càng nhỏ càng tốt).
- $b(i)$: Khoảng cách trung bình từ điểm i tới tất cả các điểm trong cụm láng giềng gần nhất tiếp theo (độ phân tách liên cụm, muốn càng lớn càng tốt).
- $s(i) \rightarrow +1$: Điểm nằm sâu trong cụm phù hợp; $s(i) \approx 0$: Điểm nằm ngay ranh giới giữa 2 cụm; $s(i) < 0$: Điểm đã bị gán sai cụm.

§1.9 Kỹ thuật Xử lý Mất Cân Bằng Lớp (Imbalanced Data)

1. **SMOTE (Synthetic Minority Over-sampling Technique):** Không sao chép trùng lặp, mà **nội suy sinh mẫu mới** trên đoạn thẳng nối mẫu thiểu số $\mathbf{x}_i$ với 1 láng giềng gần nhất $\mathbf{x}_{zi}$ cùng lớp:

$$\mathbf{x}_{\text{new}} = \mathbf{x}_i + \lambda(\mathbf{x}_{zi} - \mathbf{x}_i), \quad \lambda \sim U(0, 1) \quad (19)$$

2. **Class Weighting / Cost-Sensitive Learning:** Nhân trọng số phạt lớn hơn vào hàm mất mát khi mô hình đoán sai mẫu thiểu số: $w_c = \frac{N}{C \cdot N_c}$.
3. **Focal Loss:** Giảm thiểu hàm phạt trên các mẫu dễ phân loại, dồn toàn bộ sự chú ý của gradient vào các mẫu thiểu số khó.

§2 Deep Learning Cơ Bản & PyTorch Framework

§2.1 Perceptron, MLP & Định Lý Xấp Xỉ Phổ Quát

Mạng Perceptron Đa tầng (Multi-Layer Perceptron — MLP) ánh xạ đầu vào $\mathbf{x} \in \mathbb{R}^d$ qua các tầng ẩn:

$$\mathbf{h}^{(1)} = \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1), \quad \mathbf{h}^{(2)} = \sigma(\mathbf{W}_2 \mathbf{h}^{(1)} + \mathbf{b}_2), \quad \dots, \quad \hat{\mathbf{y}} = g(\mathbf{W}_L \mathbf{h}^{(L-1)} + \mathbf{b}_L) \quad (20)$$

- $\mathbf{W}_l$: Ma trận trọng số tại tầng l .
- $\mathbf{b}_l$: Vector độ lệch bias tại tầng l .
- $\sigma(\cdot)$: Hàm kích hoạt phi tuyến (Non-linear Activation Function).
- **Định lý Xấp xỉ Phổ quát (Cybenko, 1989):** Một mạng nơ-ron chỉ cần duy nhất 1 tầng ẩn với số lượng neuron hữu hạn và hàm kích hoạt phi tuyến liên tục có khả năng xấp xỉ bất kỳ hàm liên tục nào trên không gian compact với độ chính xác tùy ý. Tuy nhiên, việc tăng chiều sâu mạng (Deep Architecture) giúp giảm số lượng neuron theo cấp số mũ so với mạng chỉ mở rộng chiều ngang (Wide Architecture).
- **Nếu không có hàm kích hoạt phi tuyến:** Mạng sâu L tầng sẽ thu gọn thành một phép biến đổi tuyến tính đơn lẻ: $\mathbf{y} = \mathbf{W}_L \dots \mathbf{W}_1 \mathbf{x} = \mathbf{W}_{\text{eff}} \mathbf{x} + \mathbf{b}_{\text{eff}}$, mất hoàn toàn khả năng giải quyết các bài toán phi tuyến (như cổng XOR).

§2.2 Phân Tích Chi Tiết Các Hàm Kích Hoạt (Activation Functions)

- **Sigmoid:** $\sigma(z) = \frac{1}{1+e^{-z}}$. Đạo hàm: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.
 - Đạo hàm cực đại chỉ bằng 0.25 tại $z = 0$, tiệm cận 0 khi $|z|$ lớn.
 - **Nhược điểm:** Gây triệt tiêu gradient nghiêm trọng (Vanishing Gradient) trong mạng nhiều tầng; đầu ra không đối xứng quanh 0 (Not zero-centered), làm cập nhật gradient bị zig-zag.
- **Tanh (Hyperbolic Tangent):** $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$. Đạo hàm: $\tanh'(z) = 1 - \tanh^2(z) \leq 1$.
 - Miền giá trị $(-1, 1)$, đối xứng quanh 0 (Zero-centered). Hội tụ nhanh hơn Sigmoid nhưng vẫn bị hiện tượng bão hòa ở hai đầu biên.
- **ReLU (Rectified Linear Unit):** $f(z) = \max(0, z)$. Đạo hàm: $f'(z) = 1$ khi $z > 0$; $f'(z) = 0$ khi $z \leq 0$.
 - Tốc độ tính toán siêu nhanh, gradient bằng đúng 1 ở miền dương giúp giảm thiểu triệt tiêu gradient.
 - **Nhược điểm: Dying ReLU.** Nếu một neuron nhận trọng số khiến $z \leq 0$ trên toàn bộ tập dữ liệu, gradient của nó vĩnh viễn bằng 0 và neuron đó "chết" hoàn toàn, không thể cập nhật tiếp.
- **Leaky ReLU:** $f(z) = \max(\alpha z, z)$ với $\alpha \approx 0.01$. Khắc phục hiện tượng Dying ReLU nhờ duy trì một độ dốc nhỏ α ở vùng âm.
- **GELU (Gaussian Error Linear Unit):** $f(z) = z\Phi(z) = z \cdot P(X \leq z)$ với $X \sim \mathcal{N}(0, 1)$. Kích hoạt mượt mà phi tuyến, là chuẩn mực trong các mô hình Transformer hiện đại (BERT, GPT-3, ViT).

§2.3 Quy Trình Huấn Luyện Chuẩn PyTorch (Training Loop)

```
model.train() # 1. Bật chế độ train (bật Dropout, BatchNorm cap nhat running stats)
for inputs, targets in dataloader:
    optimizer.zero_grad() # 2. XÓA GRADIENT CU (mặc định PyTorch cong don)
    outputs = model(inputs) # 3. Forward pass (tính dự đoán)
    loss = criterion(outputs, targets) # 4. Tính giá trị hàm mất mát Loss
    loss.backward() # 5. Backward pass (tính đạo hàm riêng lưu vào param.grad)
    optimizer.step() # 6. Cập nhật trọng số theo thuật toán tối ưu
```

[BẮY THI CẦN TRÁNH] — Tại sao bắt buộc phải gọi `optimizer.zero_grad()` trước `loss.backward()`?

Trong PyTorch, nhằm tối ưu bộ nhớ và phục vụ kỹ thuật **Tích lũy Gradient (Gradient Accumulation)** khi GPU có VRAM nhỏ, hàm `backward()` mặc định thực hiện phép toán cộng dồn:

$$\text{param.grad} \leftarrow \text{param.grad} + \frac{\partial \mathcal{L}_{\text{batch}}}{\partial \mathbf{w}}$$

Nếu không gọi `optimizer.zero_grad()`, gradient của batch hiện tại sẽ bị cộng dồn chồng chất với toàn bộ các batch trước đó, dẫn đến bước cập nhật trọng số khổng lồ, làm bùng nổ gradient và phá hủy toàn bộ quá trình hội tụ của mô hình!

§2.4 Các Hàm Mất Mát (Loss Functions) & Bẫy Double-Softmax

- **Binary Cross-Entropy (BCE):** $\mathcal{L} = -\frac{1}{N} \sum [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$, nhận đầu vào là xác suất $\hat{y}_i \in (0, 1)$.
- **`nn.BCEWithLogitsLoss`:** Tích hợp Sigmoid và BCE bằng kỹ thuật toán học Log-Sum-Exp ổn định số:

$$\mathcal{L}(z, y) = \max(z, 0) - zy + \log(1 + e^{-|z|}) \quad (21)$$

⇒ Nhận đầu vào là **Logit thô z (chưa qua Sigmoid)**, triệt tiêu hoàn toàn nguy cơ tràn số (numerical overflow/underflow).

- **`nn.CrossEntropyLoss` (Đa lớp):** Tích hợp gộp `nn.LogSoftmax()` và `nn.NLLLoss()`:

$$\mathcal{L}_{\text{CE}}(\mathbf{z}, y) = -\log \left(\frac{e^{z_y}}{\sum_{j=1}^C e^{z_j}} \right) = -z_y + \log \left(\sum_{j=1}^C e^{z_j} \right) \quad (22)$$

- $\mathbf{z} = (z_1, z_2, \dots, z_C) \in \mathbb{R}^C$: Vector Logit thô đầu ra từ tầng Linear cuối cùng.
- $y \in \{1, 2, \dots, C\}$: Nhãn số nguyên của lớp ground-truth thực tế.
- z_y : Giá trị logit tại vị trí lớp đúng y .
- ⇒ Nhận đầu vào là **Logit thô z (CHƯA qua Softmax)**.

- **Focal Loss (Lin et al., 2017):** Xử lý mất cân bằng lớp cực đoan:

$$\text{FL}(p_t) = -\alpha_t (1 - p_t)^\gamma \log(p_t) \quad (23)$$

Trong đó p_t là xác suất dự đoán cho lớp đúng, $\gamma \geq 0$ là siêu tham số focusing. Khi mẫu dễ phân loại ($p_t \approx 0.9$), thừa số $(1 - p_t)^\gamma = (0.1)^\gamma = 0.01$, làm triệt tiêu 99% trọng số mất mát của mẫu dễ, ép gradient tập trung vào các mẫu khó.

[BẮY THI CẦN TRÁNH] — Lỗi Double-Softmax trong PyTorch

Nếu tầng cuối của mạng nơ-ron khai báo `nn.Softmax()` hoặc gọi `torch.softmax(out, dim=-1)` trước khi truyền vào `nn.CrossEntropyLoss()`, mô hình sẽ bị áp dụng hàm Softmax 2 lần liên tiếp. Khi đó xác suất đầu vào bị nén phẳng, gradient tính ra bị sai lệch trầm trọng và làm mô hình không thể hội tụ.

§2.5 Thuật Toán Tối Ưu Hóa: SGD, Momentum, RMSProp & Adam

Adam (Adaptive Moment Estimation): Khởi tạo $\mathbf{m}_0 = \mathbf{0}, \mathbf{v}_0 = \mathbf{0}, t = 0$. Tại bước lặp thứ t :

$$\mathbf{g}_t = \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}_t) \quad (24)$$

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t \quad (\text{Moment bậc 1 — Hướng di chuyển trung bình có quán tính}) \quad (25)$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2 \quad (\text{Moment bậc 2 — Độ biến động bình phương trung bình}) \quad (26)$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t} \quad (\text{Hiệu chỉnh độ chệch khởi tạo — Bias Correction}) \quad (27)$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\eta}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \odot \hat{\mathbf{m}}_t \quad (\text{Cập nhật vector trọng số mô hình}) \quad (28)$$

Giải thích chi tiết từng biến số và tham số trong thuật toán Adam:

- $t \in \{1, 2, 3, \dots\}$: Chỉ số bước lặp tối ưu hóa (Iteration / Step counter).
- $\mathbf{w}_t \in \mathbb{R}^P$: Vector toàn bộ P tham số trọng số của mô hình tại bước t .
- $\mathcal{L}(\mathbf{w}_t)$: Giá trị hàm mất mát tính trên batch dữ liệu hiện tại.
- $\mathbf{g}_t \in \mathbb{R}^P$: Vector Gradient bậc 1 của hàm mất mát theo trọng số: $\mathbf{g}_t = \left[\frac{\partial \mathcal{L}}{\partial w_1}, \dots, \frac{\partial \mathcal{L}}{\partial w_P} \right]^T$.
- $\mathbf{m}_t \in \mathbb{R}^P$: Vector Moment bậc 1 (Trung bình trượt hàm mũ của gradient). Đóng vai trò như vận tốc quán tính trong vật lý, giúp vượt qua các điểm yên ngựa (Saddle Points) và cực tiểu địa phương nông.
- $\mathbf{v}_t \in \mathbb{R}^P$: Vector Moment bậc 2 (Trung bình trượt hàm mũ của bình phương gradient). Đo lường mức độ dao động mạnh hay yếu của gradient theo từng chiều tọa độ. Phép toán $\mathbf{g}_t^2$ là bình phương từng phần tử.
- $\beta_1 \in [0, 1)$: Hệ số suy giảm của moment bậc 1. Giá trị chuẩn mực công nghiệp: $\beta_1 = 0.9$.
- $\beta_2 \in [0, 1)$: Hệ số suy giảm của moment bậc 2. Giá trị chuẩn mực công nghiệp: $\beta_2 = 0.999$.
- β_1^t, β_2^t : Lũy thừa bậc t của β_1 và β_2 .
- $\hat{\mathbf{m}}_t, \hat{\mathbf{v}}_t$: Các moment đã được **Hiệu chỉnh độ chệch (Bias Correction)**. Do ban đầu $\mathbf{m}_0 = \mathbf{0}, \mathbf{v}_0 = \mathbf{0}$, trong những bước đầu tiên $\mathbf{m}_t$ và $\mathbf{v}_t$ bị co cụm thiên vị về gần 0. Chia cho $(1 - \beta^t)$ đưa kỳ vọng không chệch trở về đúng giá trị thực: $\mathbb{E}[\hat{\mathbf{m}}_t] = \mathbb{E}[\mathbf{g}_t]$. Khi $t \rightarrow \infty, 1 - \beta^t \rightarrow 1$, hiệu chỉnh này tự động mờ dần.
- η : Tốc độ học (Learning Rate), thường đặt từ 10^{-4} đến 10^{-3} .
- ϵ : Hằng số làm mịn dương cực nhỏ (thường chọn 10^{-8}) nằm ở mẫu số để triệt tiêu lỗi chia cho 0 khi độ biến động $\hat{\mathbf{v}}_t$ quá nhỏ.
- $\odot$: Phép nhân Hadamard (Element-wise multiplication — nhân từng phần tử tương ứng).

§2.6 Khởi Tạo Trọng Số (Weight Initialization)

- **Khởi tạo Zeros (Tất cả bằng 0): HOÀN TOÀN SAI.** Mọi neuron trong cùng một lớp nhận tín hiệu giống nhau và có gradient y hệt nhau $\Rightarrow$ không phá vỡ tính đối xứng (Symmetry Problem), khiến mạng sâu bị thoái hóa về 1 neuron đơn lẻ.
- **Xavier / Glorot Initialization (Glorot & Bengio, 2010):**

$$\text{Var}(W) = \frac{2}{n_{\text{in}} + n_{\text{out}}} \quad \Rightarrow \quad \text{Bắt buộc dùng cho Tanh và Sigmoid} \quad (29)$$

n_{in} là số lượng kết nối vào (fan-in), n_{out} là số lượng kết nối ra (fan-out). Giữ cho phương sai tín hiệu không bị bùng nổ hay triệt tiêu qua các hàm kích hoạt đối xứng quanh 0.

- **He / Kaiming Initialization (He et al., 2015):**

$$\text{Var}(W) = \frac{2}{n_{\text{in}}} \quad \Rightarrow \quad \text{Bắt buộc dùng cho ReLU và Leaky ReLU} \quad (30)$$

Do ReLU triệt tiêu một nửa tín hiệu âm ($z \leq 0$), phương sai của trọng số cần phải tăng gấp đôi lên $\frac{2}{n_{\text{in}}}$ để bù đắp phần năng lượng tín hiệu bị mất.

§2.7 Batch Normalization vs Layer Normalization

Batch Normalization (Ioffe & Szegedy, 2015): Chuẩn hóa trên mini-batch $\mathcal{B} = \{x_1, \dots, x_m\}$:

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i, \quad \sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2, \quad \hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta \quad (31)$$

- m : Kích thước mini-batch (batch size).
- μ_B, σ_B^2 : Giá trị trung bình và phương sai tính toán trên mini-batch hiện tại.
- $\epsilon > 0$: Hằng số chống chia cho 0 (thường là 10^{-5}).
- γ (Scale) và β (Shift): **Hai tham số học được (Learnable parameters)** được tối ưu qua backpropagation. Giúp mạng có năng lực khôi phục lại phân phối gốc nếu việc ép về chuẩn hóa làm suy giảm năng lực biểu diễn phi tuyến.
- Thứ tự chuẩn trong kiến trúc mạng: **Linear / Conv $\rightarrow$ BatchNorm $\rightarrow$ ReLU**.
- Trong pha kiểm thử (Inference): Sử dụng giá trị trung bình tích lũy `running_mean` và phương sai `running_var` tính bằng trung bình trượt trong quá trình train.
- **Layer Normalization (Ba et al., 2016):** Chuẩn hóa độc lập trên từng mẫu riêng lẻ dọc theo chiều đặc trưng (Features). Không phụ thuộc batch size $\implies$ **Chuẩn mực bắt buộc cho Transformer, RNN và dữ liệu chuỗi NLP**.

§2.8 Cơ Chế Inverted Dropout

Trong pha Train, ngẫu nhiên ngắt neuron với xác suất ngắt p , đồng thời nhân hệ số phóng đại $\frac{1}{1-p}$:

$$\mathbf{h}_{\text{train}} = \frac{1}{1-p} (\mathbf{h} \odot \mathbf{m}), \quad \mathbf{m} \sim \text{Bernoulli}(1-p) \quad (32)$$

- $\mathbf{h} \in \mathbb{R}^d$: Vector kích hoạt đầu ra của một tầng.
- $p \in [0, 1]$: Xác suất neuron bị loại bỏ (Drop probability, thường đặt 0.5 cho FC layer và 0.1 cho Attention).
- $\mathbf{m} \in \{0, 1\}^d$: Mặt nạ nhị phân ngẫu nhiên (Binary mask) tuân theo phân phối Bernoulli với xác suất giữ lại là $1-p$.
- Thừa số $\frac{1}{1-p}$: Đảm bảo kỳ vọng toán học của tín hiệu không thay đổi: $\mathbb{E}[\mathbf{h}_{\text{train}}] = \mathbf{h}$. Nhờ vậy, trong pha Test/Inference, ta chỉ cần tắt Dropout (`model.eval()`) và chuyển thẳng $\mathbf{h}_{\text{test}} = \mathbf{h}$ mà không cần nhân lại trọng số.

§3 Thị Giác Máy Tính (Computer Vision)

§3.1 Công Thức Kích Thước Không Gian Tầng Conv2D

$$O = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1 \quad (33)$$

Giải thích chi tiết các ký hiệu:

- W : Kích thước chiều không gian đầu vào (Chiều rộng W hoặc Chiều cao H).
- K : Kích thước cạnh của bộ lọc tích chập (Kernel / Filter size, ví dụ kernel $3 \times 3 \implies K = 3$).
- P : Độ dày viền chèn thêm xung quanh ma trận (Padding).
- S : Bước nhảy trượt của kernel qua mỗi lần tính tích chập (Stride).
- O : Kích thước chiều không gian đầu ra (Output feature map dimension).
- $\lfloor \cdot \rfloor$: Phép lấy phần nguyên sàn (Floor).

Các chế độ Padding phổ biến:

- **Valid Padding ($P = 0$):** Không thêm viền $\implies O = \lfloor \frac{W-K}{S} \rfloor + 1$. Kích thước luôn bị thu nhỏ sau phép tích chập.
- **Same Padding ($S = 1$):** Chọn $P = \frac{K-1}{2}$ (khi K lẻ) để kích thước đầu ra giữ nguyên bằng đầu vào: $O = W$.

Công thức tính Tổng số lượng tham số học được (Learnable Parameters):

$$\text{Params} = (K_H \times K_W \times C_{\text{in}} + 1) \times C_{\text{out}} \quad (34)$$

- K_H, K_W : Chiều cao và chiều rộng của nhân kernel (ví dụ 3×3).
- C_{in} : Số kênh của ảnh hoặc feature map đầu vào (ví dụ ảnh RGB thì $C_{\text{in}} = 3$).
- $+1$: Tham số Bias tương ứng cho mỗi bộ lọc (Filter).
- C_{out} : Số lượng bộ lọc kernel, đồng thời là số kênh của feature map đầu ra.

[VÍ DỤ TÍNH TOÁN TAY] — Tính toán kích thước Conv2D thường gặp trong đề thi

1. Ảnh 32×32 , kernel 3×3 , stride 1, padding 1: $O = \lfloor \frac{32-3+2(1)}{1} \rfloor + 1 = 31 + 1 = 32 \implies 32 \times 32$.
2. Ảnh 28×28 , kernel 5×5 , stride 1, không padding ($P = 0$): $O = \lfloor \frac{28-5+0}{1} \rfloor + 1 = 23 + 1 = 24 \implies 24 \times 24$.
3. Ảnh 224×224 , kernel 7×7 , stride 2, padding 3: $O = \lfloor \frac{224-7+2(3)}{2} \rfloor + 1 = \lfloor \frac{223}{2} \rfloor + 1 = 111 + 1 = 112 \implies 112 \times 112$.
4. Tính số tham số: Lớp Conv2D nhận đầu vào 64 kênh, xuất ra 128 kênh, kernel 3×3 : $\text{Params} = (3 \times 3 \times 64 + 1) \times 128 = (576 + 1) \times 128 = 577 \times 128 = 73\,856$ tham số.

§3.2 Pooling, Global Average Pooling (GAP) & Receptive Field

- **Pooling:** Max Pooling giữ đặc trưng kích hoạt mạnh nhất; Average Pooling làm mượt. **Lớp Pooling hoàn toàn không chứa tham số học được (0 learnable parameters).**
- **Global Average Pooling (GAP):** Ép toàn bộ không gian $H \times W$ của mỗi channel thành 1 giá trị vô hướng duy nhất ($H \times W \times C \rightarrow 1 \times 1 \times C$). Thay thế hoàn toàn lớp Fully Connected công kênh, giảm hàng triệu tham số và chống Overfitting mạnh mẽ.
- **Receptive Field (RF — Vùng cảm thụ):** Kích thước vùng trên ảnh gốc tác động tới giá trị kích hoạt của một neuron ở tầng sâu. Công thức tích lũy qua tầng l :

$$RF_l = RF_{l-1} + (K_l - 1) \cdot \prod_{i=1}^{l-1} S_i \quad (35)$$

Xếp chồng hai lớp 3×3 với stride 1 liên tiếp tạo ra $RF = 1 + (3 - 1) + (3 - 1) = 5 \times 5$, nhưng số tham số chỉ là $2 \times (3^2) = 18$ (so với $5^2 = 25$ của một tầng 5×5 đơn lẻ), đồng thời bổ sung thêm 2 lần kích hoạt phi tuyến ReLU.

§3.3 Kiến Trúc CNN Kinh Điển & Đột Phá ResNet

Cơ chế Residual Learning của ResNet (He et al., 2015): Thay vì ép mạng học trực tiếp hàm ánh xạ $\mathcal{H}(\mathbf{x})$, mạng học phần dư $\mathcal{F}(\mathbf{x}) = \mathcal{H}(\mathbf{x}) - \mathbf{x}$:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x} \quad (36)$$

Khi lan truyền ngược tính gradient theo đầu vào $\mathbf{x}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \left(\frac{\partial \mathcal{F}}{\partial \mathbf{x}} + \mathbf{I} \right) \quad (37)$$

Nhờ số hạng ma trận đơn vị $\mathbf{I}$, gradient luôn có một "đường cao tốc" chảy thẳng ngược về các tầng nông đầu tiên mà không bao giờ bị triệt tiêu hoàn toàn, giải quyết triệt để hiện tượng thoái hóa mô hình (Degradation Problem) khi đào sâu mạng lên tới 152 tầng.

§3.4 Phân Biệt Skip Connection: ResNet (ADD) vs U-Net (CONCAT)

- **ResNet (ADD):** Thực hiện phép **CỘNG theo từng phần tử (Element-wise ADD)**. Yêu cầu số kênh và kích thước không gian hai nhánh phải giống nhau. Mục tiêu: Học phần dư để đào sâu mạng.
- **U-Net (CONCAT):** Thực hiện phép **GHÉP NỐI theo chiều kênh (Channel-wise CONCAT)**. Ghép trực tiếp feature map độ phân giải cao từ Encoder sang Decoder. Mục tiêu: Giữ lại chi tiết không gian chính xác phục vụ bài toán Phân vùng ảnh (Segmentation).

§3.5 Object Detection: IoU, NMS, mAP, YOLO vs Faster R-CNN

Intersection over Union (IoU):

$$\text{IoU} = \frac{\text{Diện tích vùng Giao}}{\text{Diện tích vùng Hợp}} = \frac{|B_{\text{pred}} \cap B_{\text{gt}}|}{|B_{\text{pred}} \cup B_{\text{gt}}|} \quad (38)$$

- B_{pred} : Bounding box do mô hình dự đoán.
- B_{gt} : Bounding box nhãn thực tế (Ground Truth).
- Một dự đoán được công nhận là True Positive (TP) nếu $\text{IoU} \geq$ ngưỡng (thường là 0.5) và đúng nhãn lớp.

Thuật toán Non-Maximum Suppression (NMS) 4 bước:

1. Loại bỏ toàn bộ các box có Confidence Score nhỏ hơn ngưỡng tin cậy T_{conf} (ví dụ 0.25).
2. Chọn box B_{max} có Confidence Score cao nhất trong danh sách còn lại, lưu vào danh sách giữ lại.
3. Tính IoU giữa B_{max} và tất cả các box còn lại trong danh sách. Xóa bỏ toàn bộ các box có $\text{IoU} \geq T_{\text{NMS}}$ (thường đặt 0.5) vì chúng cùng dự đoán trùng lặp một vật thể.
4. Lặp lại bước 2 và 3 cho đến khi không còn box nào trong danh sách ứng viên.

Độ đo mAP (Mean Average Precision):

$$\text{AP} = \int_0^1 P(R) dR, \quad \text{mAP} = \frac{1}{C} \sum_{c=1}^C \text{AP}_c \quad (39)$$

- AP: Diện tích dưới đường cong Precision-Recall của một lớp.
- $\text{mAP}@0.5$: Giá trị mAP tính tại ngưỡng IoU = 0.5.
- $\text{mAP}@[0.5:0.95]$: Trung bình cộng mAP tại 10 mức ngưỡng IoU từ 0.50 đến 0.95 với bước nhảy 0.05 (chuẩn benchmark COCO).

§4 Xử Lý Ngôn Ngữ Tự Nhiên (Natural Language Processing)

§4.1 Pipeline Tiền Xử Lý Văn Bản Chuẩn 5 Bước

1. **Tokenization:** Tách chuỗi văn bản thành các đơn vị cơ sở (Word, Subword: BPE, WordPiece).
2. **Normalization:** Chuyển chữ thường, chuẩn hóa dấu tiếng Việt (NFC vs NFD), xóa URL/HTML/ký tự lạ.
3. **Lemmatization vs Stemming:** Stemming chặt đuôi từ bằng quy tắc cơ học (bị lỗi chính tả); Lemmatization đưa về từ nguyên mẫu theo từ điển và ngữ pháp.
4. **POS Tagging:** Gán nhãn từ loại (Danh từ, Động từ, Tính từ).
5. **Stopwords Removal:** Loại bỏ các từ dừng có tần suất cao nhưng ít giá trị thông tin.

§4.2 Các Phương Pháp Biểu Diễn Từ (Word Representations)

- **TF-IDF**: Tần suất từ $TF(t, d)$ nhân với nghịch đảo tần suất tài liệu $IDF(t, D) = \log \left(\frac{|D|}{|\{d \in D: t \in d\}|} \right)$.
- **Word2Vec (Mikolov et al., 2013)**: Gồm 2 kiến trúc: **CBOW** (dự đoán từ trung tâm dựa vào ngữ cảnh xung quanh) và **Skip-gram** (dự đoán ngữ cảnh xung quanh dựa vào từ trung tâm, kết hợp Negative Sampling).
- **FastText (Bojanowski et al., 2017)**: Biểu diễn từ bằng tập hợp các ký tự **n-gram (Subwords)**. Giải quyết triệt để vấn đề Từ ngoài từ điển (OOV — Out-of-Vocabulary).

§4.3 Độ Đo Cosine Similarity

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2} = \frac{\sum_{i=1}^d u_i v_i}{\sqrt{\sum_{i=1}^d u_i^2} \sqrt{\sum_{i=1}^d v_i^2}} \in [-1, 1] \quad (40)$$

- $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$: Hai vector nhúng embedding cần so khớp độ tương đồng ngữ nghĩa.
- $\mathbf{u} \cdot \mathbf{v}$: Tích vô hướng (Dot product) giữa hai vector.
- $\|\mathbf{u}\|_2, \|\mathbf{v}\|_2$: Chuẩn Euclid (độ dài hình học) của các vector.
- **Bản chất**: Chỉ đo lường góc nghiêng giữa hai vector, triệt tiêu hoàn toàn ảnh hưởng của độ dài văn bản $\Rightarrow$ **Chuẩn mực bắt buộc trong Information Retrieval và Vector Search**.

§4.4 Mạng Hồi Quy Tuần Tự: RNN, LSTM & GRU

§4.4.1 Mạng Nơ-ron Hồi Quy Cổ Điển (Vanilla RNN)

Với chuỗi đầu vào $(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T)$, tại mỗi bước thời gian $t \in \{1, \dots, T\}$:

$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh} \mathbf{h}_{t-1} + \mathbf{W}_{xh} \mathbf{x}_t + \mathbf{b}_h) \quad (41)$$

Giải thích chi tiết các ký hiệu:

- t : Chỉ số bước thời gian (Time step), ứng với vị trí của từ thứ t trong câu.
- $\mathbf{x}_t \in \mathbb{R}^{d_{in}}$: Vector đầu vào tại thời điểm t (vector word embedding của từ thứ t).
- $\mathbf{h}_t \in \mathbb{R}^{d_h}$: Vector trạng thái ẩn (Hidden state) tại thời điểm t , đóng vai trò trí nhớ ngắn hạn tổng hợp ngữ cảnh từ bước 1 đến bước t .
- $\mathbf{h}_{t-1} \in \mathbb{R}^{d_h}$: Vector trạng thái ẩn được truyền lại từ bước liền trước $t - 1$ ($\mathbf{h}_0 = \mathbf{0}$).
- $\mathbf{W}_{hh} \in \mathbb{R}^{d_h \times d_h}$: Ma trận trọng số kết nối giữa ẩn quá khứ và ẩn hiện tại (dùng chung qua mọi bước).
- $\mathbf{W}_{xh} \in \mathbb{R}^{d_h \times d_{in}}$: Ma trận trọng số kết nối giữa đầu vào hiện tại và trạng thái ẩn.
- $\mathbf{b}_h \in \mathbb{R}^{d_h}$: Vector bias độ lệch.
- **Nhược điểm chí mạng**: Khi chuỗi dài, phép nhân ma trận liên tiếp $\prod \mathbf{W}_{hh}$ khi lan truyền ngược qua thời gian (BPTT) khiến gradient bị co về 0 theo hàm mũ $\Rightarrow$ **Vanishing Gradient**, mất khả năng ghi nhớ dài hạn.

§4.4.2 Mạng LSTM (Long Short-Term Memory — Hochreiter & Schmidhuber, 1997)

LSTM bổ sung đường truyền **Cell State ($\mathbf{C}_t$)** chạy xuyên suốt như một "băng chuyền trí nhớ dài hạn", được điều tiết bởi 3 cổng (Gates) sử dụng hàm kích hoạt Sigmoid $\sigma \in (0, 1)$:

$$1. \text{ Cổng Quên (Forget Gate): } \mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad (42)$$

$$2. \text{ Cổng Nhớ (Input Gate): } \mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad (43)$$

$$\text{Ứng viên Cell State mới: } \tilde{\mathbf{C}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) \quad (44)$$

$$3. \text{ Cập nhật Cell State Băng chuyền: } \mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t \quad (45)$$

$$4. \text{ Cổng Xuất (Output Gate): } \mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \quad (46)$$

$$\text{Trạng thái ẩn đầu ra: } \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t) \quad (47)$$

Mở xẻ chi tiết từng tham số và ký hiệu toán học:

- t : Chỉ số thời gian (bước thứ t , từ thứ t).
- $\mathbf{x}_t \in \mathbb{R}^{d_{in}}$: Vector đầu vào của từ tại vị trí t .
- $\mathbf{h}_{t-1} \in \mathbb{R}^{d_h}$: Vector trạng thái ẩn (ngữ cảnh ngắn hạn) từ bước trước truyền sang.
- $[\mathbf{h}_{t-1}, \mathbf{x}_t] \in \mathbb{R}^{d_h+d_{in}}$: Phép ghép nối vector (Concatenation) đặt hai vector nằm cạnh nhau.
- $\mathbf{f}_t \in (0, 1)^{d_h}$: Vector **Cổng Quên**. Đi qua hàm σ nên các phần tử mang giá trị từ 0 đến 1. Giá trị tiến về 0 nghĩa là "quên sạch thông tin cũ tương ứng", tiến về 1 nghĩa là "bảo tồn hoàn toàn thông tin cũ".
- $\mathbf{i}_t \in (0, 1)^{d_h}$: Vector **Cổng Nhớ**, quyết định mức độ bao nhiêu phần trăm thông tin mới được phép ghi vào bộ nhớ dài hạn.
- $\tilde{\mathbf{C}}_t \in (-1, 1)^{d_h}$: Vector **Ứng viên Cell State mới**, chứa lượng thông tin mới tinh chất được chắt lọc qua hàm tanh.
- $\mathbf{C}_t \in \mathbb{R}^{d_h}$: Vector **Cell State (Trạng thái ô / Bộ nhớ dài hạn)** tại bước t . Phép toán $\mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t$ là phép tổ hợp tuyến tính: phần quá khứ được giữ lại cộng với phần kiến thức mới được nạp vào.
- $\mathbf{o}_t \in (0, 1)^{d_h}$: Vector **Cổng Xuất**, quyết định phần nào của bộ nhớ dài hạn $\mathbf{C}_t$ sẽ được bộc lộ ra làm trạng thái ẩn $\mathbf{h}_t$.
- $\mathbf{h}_t \in (-1, 1)^{d_h}$: Trạng thái ẩn đầu ra tại bước t , vừa dùng để đưa vào tầng phân loại, vừa truyền sang bước tiếp theo $t + 1$.
- $\mathbf{W}_f, \mathbf{W}_i, \mathbf{W}_c, \mathbf{W}_o \in \mathbb{R}^{d_h \times (d_h+d_{in})}$: Các ma trận trọng số huấn luyện của từng cổng tương ứng.
- $\mathbf{b}_f, \mathbf{b}_i, \mathbf{b}_c, \mathbf{b}_o \in \mathbb{R}^{d_h}$: Các vector bias độ lệch tương ứng với từng cổng.
- $\odot$: Phép nhân Hadamard (Element-wise multiplication — nhân từng phần tử tương ứng giữa hai vector cùng số chiều).
- σ : Hàm Sigmoid nén giá trị về $(0, 1)$, đóng vai trò công tắc van điều tiết tỷ lệ đóng mở cổng.

[BẮY THI CẦN TRÁNH] — Tại sao LSTM triệt tiêu được Vanishing Gradient?

Đạo hàm riêng của băng chuyển Cell State: $\frac{\partial \mathbf{C}_t}{\partial \mathbf{C}_{t-1}} = \mathbf{f}_t$. Khi mô hình học được việc duy trì cổng quên mở ($\mathbf{f}_t \approx 1$), gradient lan truyền ngược dọc theo $\mathbf{C}$ sẽ được nhân với số 1, bảo toàn nguyên vẹn độ lớn qua hàng trăm bước thời gian mà không bị suy giảm theo cấp số nhân như phép nhân ma trận trọng số $\mathbf{W}_{hh}$ trong Vanilla RNN!

§4.4.3 Mạng GRU (Gated Recurrent Unit — Cho et al., 2014)

GRU rút gọn cấu trúc còn 2 cổng, gộp chung Cell State vào Hidden State $\mathbf{h}_t$:

$$\text{Cổng Tái lập (Reset Gate): } \mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r) \quad (48)$$

$$\text{Cổng Cập nhật (Update Gate): } \mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z) \quad (49)$$

$$\text{Trạng thái ứng viên: } \tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_h) \quad (50)$$

$$\text{Cập nhật Trạng thái ẩn: } \mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \quad (51)$$

- $\mathbf{z}_t$: Gộp chức năng của cả Cổng Quên và Cổng Nhớ trong LSTM: thông tin cũ giữ lại là $(1 - \mathbf{z}_t)$, thông tin mới nạp vào là $\mathbf{z}_t$.
- Giảm khoảng 25% số lượng tham số so với LSTM, tốc độ huấn luyện nhanh hơn trên các tập dữ liệu kích thước vừa và nhỏ.

§4.5 Kiến Trúc Transformer (Vaswani et al., 2017)**Scaled Dot-Product Attention:**

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (52)$$

Giải thích bản chất chi tiết của từng thành phần:

- $Q \in \mathbb{R}^{N \times d_k}$ (Query Matrix): Ma trận truy vấn. Mỗi hàng đại diện cho một từ đang "đi tìm kiếm" thông tin liên quan từ các từ khác. Tạo bởi $Q = XW^Q$.
- $K \in \mathbb{R}^{M \times d_k}$ (Key Matrix): Ma trận chìa khóa. Mỗi hàng đóng vai trò như nhãn định danh / tiêu đề của từng từ để khớp với Query. Tạo bởi $K = XW^K$.
- $V \in \mathbb{R}^{M \times d_v}$ (Value Matrix): Ma trận giá trị. Mỗi hàng chứa đựng nội dung ngữ nghĩa thực sự mà từ đó mang theo. Tạo bởi $V = XW^V$.
- d_k : Số chiều của vector Query và Key (trong Transformer gốc, $d_{\text{model}} = 512, h = 8 \implies d_k = 64$).
- $QK^T \in \mathbb{R}^{N \times M}$: Ma trận tích vô hướng đo độ tương đồng ngữ nghĩa giữa mọi cặp từ.
- $\text{softmax}(\cdot)$: Chuẩn hóa từng hàng thành phân phối xác suất tổng bằng 1 (Attention Weights — Trọng số chú ý).
- Nhân với V : Tính tổng có trọng số của các vector Value, cho ra biểu diễn ngữ cảnh hoàn toàn mới tích hợp thông tin từ toàn bộ câu.

[BẮY THI CẦN TRÁNH] — Tại sao bắt buộc phải chia cho $\sqrt{d_k}$?

Giả sử các phần tử của Q và K là các biến ngẫu nhiên độc lập có kỳ vọng bằng 0 và phương sai bằng 1. Khi đó tích vô hướng $q \cdot k = \sum_{j=1}^{d_k} q_j k_j$ có phương sai bằng đúng d_k . Khi d_k lớn (ví dụ $d_k = 64$), giá trị tích vô hướng trở nên cực lớn, đẩy hàm Softmax vào vùng bão hòa cực đoan nơi đạo hàm tiệm cận về 0 $\implies$ **Vanishing Gradient**. Phép chia cho $\sqrt{d_k}$ kéo phương sai trở về 1, giữ cho gradient ổn định.

Positional Encoding (Sinusoidal): Do cơ chế Self-Attention xử lý toàn bộ các từ song song và có tính chất bất biến hoán vị (Permutation Invariant), bắt buộc phải cộng vector vị trí vào embedding:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (53)$$

- pos : Vị trí thứ tự của từ trong câu ($pos = 0, 1, 2, \dots$).
- i : Chỉ số chiều trong không gian vector embedding ($i \in [0, d_{\text{model}}/2 - 1]$).
- d_{model} : Tổng số chiều của vector embedding (ví dụ 512 hoặc 768).
- Các bước sóng hình sin trải dài từ 2π đến $10000 \cdot 2\pi$, cho phép mô hình dễ dàng học cách tham chiếu các vị trí tương đối thông qua công thức lượng giác cộng góc: $PE_{pos+k} = f(PE_{pos})$.

§4.6 Các Độ Đo Đánh Giá NLP: BLEU, SacreBLEU, ROUGE & Perplexity

BLEU (Bilingual Evaluation Understudy):

$$\text{BLEU} = BP \cdot \exp\left(\sum_{n=1}^N w_n \log p_n\right), \quad BP = \min\left(1, e^{1-r/c}\right) \quad (54)$$

- p_n : Modified n-gram precision (tỉ lệ các cụm n từ trong câu sinh ra xuất hiện trong câu tham chiếu, có cắt bớt tần suất xuất hiện cực đại để tránh gian lận lặp từ).
- w_n : Trọng số của từng bậc n-gram (thường dùng $N = 4$ và $w_n = 1/4$).
- c : Độ dài của câu do mô hình sinh ra (Candidate length).
- r : Độ dài của câu dịch tham chiếu chuẩn của con người (Reference length).
- BP (Brevity Penalty): Hệ số phạt câu sinh ra quá ngắn. Nếu $c > r$, $BP = 1$; nếu $c \leq r$, $BP = e^{1-r/c} < 1$.
- **ROUGE**: Định hướng **Recall**, đo lường mức độ bao phủ thông tin, là chuẩn mực cho bài toán **Tóm tắt văn bản (Summarization)**.
- **Perplexity (PPL)**: $\text{PPL} = \exp(\mathcal{L}_{\text{CE}})$, đo lường độ bối rối của mô hình ngôn ngữ. **PPL càng thấp mô hình càng xuất sắc**.

§5 Toán & Xác Suất Thống Kê Cho Trí Tuệ Nhân Tạo

§5.1 Định Lý Bayes & Ngụy Biện Tỷ Lệ Cơ Sở (Base-Rate Fallacy)

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)} = \frac{P(B|A)P(A)}{P(B|A)P(A) + P(B|\neg A)P(\neg A)} \quad (55)$$

- $P(A)$: Xác suất Tiên nghiệm (Prior Probability) — niềm tin ban đầu khi chưa có dữ liệu.
- $P(B|A)$: Khả năng xảy ra (Likelihood) — xác suất quan sát thấy bằng chứng B khi giả thuyết A đúng.
- $P(B)$: Biên bằng chứng (Evidence / Marginal Probability) — xác suất toàn phần xuất hiện bằng chứng B .
- $P(A|B)$: Xác suất Hậu nghiệm (Posterior Probability) — niềm tin được cập nhật về A sau khi đã quan sát thấy bằng chứng B .

[VÍ DỤ TÍNH TOÁN TAY] — Bài toán Chẩn đoán Y tế Xét nghiệm Bệnh Hiếm (Base-Rate Fallacy)

Giả sử một căn bệnh hiếm có tỉ lệ mắc trong dân số là 2% ($P(\text{Bệnh}) = 0.02 \implies P(\text{Khỏe}) = 0.98$). Một xét nghiệm có độ nhạy 95% ($P(+|\text{Bệnh}) = 0.95$) và tỉ lệ dương tính giả là 4% ($P(+|\text{Khỏe}) = 0.04$). Một người ngẫu nhiên có kết quả xét nghiệm Dương tính (+). Xác suất người đó thực sự mắc bệnh là bao nhiêu?

- Tử số (Likelihood $\times$ Prior): $P(+|\text{Bệnh}) \cdot P(\text{Bệnh}) = 0.95 \times 0.02 = 0.019$.
- Mẫu số (Xác suất toàn phần $P(+)$): $P(+) = 0.019 + P(+|\text{Khỏe}) \cdot P(\text{Khỏe}) = 0.019 + (0.04 \times 0.98) = 0.019 + 0.0392 = 0.0582$.
- Xác suất hậu nghiệm thực sự mắc bệnh:

$$P(\text{Bệnh}|+) = \frac{0.019}{0.0582} \approx 32.65\%$$

Kết luận: Dù xét nghiệm có độ nhạy rất cao (95%), nhưng do tỷ lệ bệnh trong dân số quá thấp (2%), xác suất một người dương tính thực sự mắc bệnh **chỉ khoảng 32.6%**! Hơn 67% những người nhận kết quả dương tính thực chất là dương tính giả.

§5.2 Tính Chất của Kỳ Vọng, Phương Sai & Ma Trận Tương Quan

$$\mathbb{E}[aX + b] = a\mathbb{E}[X] + b \quad (56)$$

$$\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 \quad (57)$$

$$\text{Var}(aX + b) = a^2\text{Var}(X) \quad (\text{hằng số cộng } b \text{ không làm đổi độ biến động}) \quad (58)$$

Nếu X và Y là hai biến ngẫu nhiên độc lập:

$$\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y), \quad \text{Var}(X - Y) = \text{Var}(X) + \text{Var}(Y) \quad (59)$$

Hiệp phương sai (Covariance) & Hệ số tương quan Pearson:

$$\text{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])], \quad r_{XY} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y} \in [-1, 1] \quad (60)$$

§5.3 Ước Lượng Tham Số: MLE vs MAP & Mối Liên Hệ Với Regularization

- **Maximum Likelihood Estimation (MLE):** Tối đa hóa hàm hợp lý dựa thuần túy trên dữ liệu quan sát:

$$\hat{\theta}_{\text{MLE}} = \arg \max_{\theta} \sum_{i=1}^N \log P(x_i|\theta) \quad (61)$$

- **Maximum A Posteriori (MAP):** Tích hợp phân phối niềm tin tiên nghiệm (Prior) $P(\theta)$:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} \left[\sum_{i=1}^N \log P(x_i|\theta) + \log P(\theta) \right] \quad (62)$$

- **Cầu nối toán học với Regularization:**

- Tiên nghiệm phân phối Chuẩn $\theta \sim \mathcal{N}(0, \sigma_0^2) \implies \log P(\theta) = -\frac{1}{2\sigma_0^2} \|\theta\|_2^2 + \text{const} \iff$ **Regularization L2 (Ridge).**
- Tiên nghiệm phân phối Laplace $P(\theta) \propto \exp(-\lambda \|\theta\|_1) \implies \log P(\theta) = -\lambda \|\theta\|_1 \iff$ **Regularization L1 (Lasso).**

§5.4 Kiểm Định Giả Thuyết Thống Kê & Ý Nghĩa Của p-value

- Giả thuyết vô hiệu H_0 (mặc định không có sự khác biệt), Giả thuyết đối H_1 .

- **Định nghĩa chuẩn mực của p-value:**

$$p\text{-value} = P(\text{Quan sát được kết quả cực đoan như thế hoặc hơn thế} \mid H_0 \text{ đúng}) \quad (63)$$

- Nếu $p\text{-value} < \alpha$ (thường là 0.05), ta bác bỏ giả thuyết H_0 ở mức ý nghĩa thống kê α .
- **Bẫy đề thi:** $p\text{-value}$ **KHÔNG PHẢI** là xác suất giả thuyết H_0 đúng ($P(H_0|\text{Data})$); $p\text{-value}$ cũng không đo lường độ lớn hiệu ứng thực tế (Effect size).

§5.5 Đại Số Tuyến Tính Nâng Cao Cho AI: Ma Trận Trục Giao & Ma Trận Hessian

1. Ma trận Trục Giao (Orthogonal Matrix): Ma trận vuông $A \in \mathbb{R}^{n \times n}$ được gọi là ma trận trục giao nếu tích với chuyển vị của nó bằng ma trận đơn vị:

$$A^T A = A A^T = I_n \implies A^{-1} = A^T \quad (64)$$

- **Định thức:** $\det(A^T A) = \det(A)^2 = \det(I) = 1 \implies \det(A) = \pm 1 \neq 0$. Do đó A luôn khả nghịch (không bao giờ suy biến).
- **Bảo toàn tích vô hướng và độ dài vector:** $\langle A\mathbf{x}, A\mathbf{y} \rangle = (A\mathbf{x})^T (A\mathbf{y}) = \mathbf{x}^T A^T A \mathbf{y} = \mathbf{x}^T \mathbf{y} = \langle \mathbf{x}, \mathbf{y} \rangle$. Do đó $\|A\mathbf{x}\|_2 = \|\mathbf{x}\|_2$, phép biến đổi trục giao giữ nguyên khoảng cách Euclid và góc giữa các vector (ứng dụng trong phép quay và phản xạ không gian).

2. Ma trận Hessian & Điều Kiện Cực Trị Bậc Hai (Second-Order Optimality): Cho hàm số khả vi hai lần $f: \mathbb{R}^n \rightarrow \mathbb{R}$. Ma trận Hessian $H(\mathbf{x}) = \nabla^2 f(\mathbf{x}) \in \mathbb{R}^{n \times n}$ chứa toàn bộ các đạo hàm riêng bậc hai:

$$H_{ij}(\mathbf{x}) = \frac{\partial^2 f(\mathbf{x})}{\partial x_i \partial x_j} \quad (65)$$

Tại điểm dừng $\mathbf{x}^*$ (nơi gradient triệt tiêu $\nabla f(\mathbf{x}^*) = \mathbf{0}$):

- **Xác định dương** ($H \succ 0$, mọi trị riêng $\lambda_i > 0$): $\mathbf{u}^T H \mathbf{u} > 0, \forall \mathbf{u} \neq \mathbf{0} \implies \mathbf{x}^*$ là **Điểm cực tiểu địa phương (Local Minimum)**.
- **Xác định âm** ($H \prec 0$, mọi trị riêng $\lambda_i < 0$): $\mathbf{u}^T H \mathbf{u} < 0, \forall \mathbf{u} \neq \mathbf{0} \implies \mathbf{x}^*$ là **Điểm cực đại địa phương (Local Maximum)**.
- **Không xác định dấu (Indefinite, có cả $\lambda_i > 0$ và $\lambda_j < 0$):** $\mathbf{x}^*$ là **Điểm yên ngựa (Saddle Point)**.

§5.6 Kỹ Thuật Cross-Validation Cho Time-Series & Chống Rò Rỉ Dữ Liệu (Data Leakage)

[BẮY THI CẦN TRÁNH] — Tuyệt đối không dùng K-Fold ngẫu nhiên cho Dữ liệu Chuỗi thời gian

Dữ liệu chuỗi thời gian có tính phụ thuộc thời gian (Temporal Autocorrelation). Nếu dùng K-Fold ngẫu nhiên, mô hình sẽ sử dụng dữ liệu ở tương lai để dự đoán cho quá khứ (Look-ahead bias / Target leakage), dẫn đến điểm validation cao ảo nhưng mô hình sụp đổ hoàn toàn khi triển khai thực tế.

- **Chiến lược đúng:** Dùng **Time-Series Split** (Rolling Window hoặc Expanding Window), trong đó tập Train luôn diễn ra trước tập Validation theo dòng thời gian.
- **Quy tắc chống Data Leakage:** Mọi thao tác tính toán đặc trưng (Fit StandardScaler, Imputer, Target Encoding) **chỉ được thực hiện trên tập Train**, sau đó dùng các tham số đó áp dụng (Transform) lên tập Test/Validation. Tuyệt đối không gọi ‘fit’ trên toàn bộ dữ liệu trước khi chia train/test!

§6 Bảng 24 Bẫy Đề Thi Kinh Điển (Lỗi Hay Mắc: SAI → ĐÚNG)

#	Quan niệm SAI (Bẫy đề thi)	Kiến thức ĐÚNG & Cơ chế hoạt động
1	BatchNorm đặt sau hàm kích hoạt ReLU	Chuẩn mực là: Linear / Conv → BatchNorm → ReLU .
2	Thêm Softmax trước nn.CrossEntropyLoss	CrossEntropyLoss đã tích hợp sẵn LogSoftmax, nhận Logit thô .
3	Thêm Sigmoid trước nn.BCEWithLogitsLoss	BCEWithLogitsLoss đã có sẵn sigmoid bên trong, nhận Logit thô .
4	Thuật toán k-NN có giai đoạn cập nhật trọng số	k-NN là Lazy Learner , chỉ lưu dữ liệu, không có phase train.
5	$IoU = \text{Diện tích Giao} / \text{Diện tích toàn ảnh}$	$IoU = \frac{\text{Giao}}{\text{Hợp}}$ của 2 bounding box.
6	Entropy trong Decision Tree dùng $\log_{10}$ hoặc $\ln$	Lý thuyết thông tin dùng $\log_2$, đơn vị là bit .
7	Stable Diffusion là một biến thể nâng cấp của GAN	Là Diffusion Model (khử nhiễu từng bước), không phải GAN.
8	Gọi optimizer.zero_grad() sau loss.backward()	Phải gọi zero_grad() trước backward() vì grad mặc định cộng dồn.
9	Skip connection trong U-Net là phép cộng ADD	U-Net dùng phép ghép nối kênh (CONCAT) ; ADD là của ResNet.
10	Dùng Accuracy để đánh giá bài toán lệch lớp	Bị đánh lừa bởi lớp đa số; bắt buộc dùng F1-score, PR-AUC .
11	Khởi tạo toàn bộ ma trận trọng số bằng 0	Gây hiện tượng đối xứng (Symmetry) , neuron cập nhật y hệt nhau.
12	Giữ Dropout bật trong giai đoạn Test / Inference	Phải tắt Dropout khi inference (model.eval()) .
13	So sánh vector embedding bằng khoảng cách Euclid	Chuẩn là dùng Cosine Similarity (bỏ qua độ lớn norm vector).
14	p-value là xác suất giả thuyết H_0 đúng	p-value là $P(\text{dữ liệu cực đoan} \mid H_0 \text{ đúng})$.
15	Vision Transformer (ViT) dùng các lớp Conv 3×3	ViT không dùng Conv , cắt patches và đưa vào Transformer Encoder.
16	Siêu tham số γ của SVM RBF lớn thì biên mượt	γ càng lớn thì biên càng cong ôm sát từng điểm $\implies$ Overfitting .
17	Regularization L1 (Lasso) chỉ co nhỏ trọng số	L1 triệt tiêu trọng số về chính xác bằng 0 $\implies$ Tạo tính thưa (Sparsity) .

#	Quan niệm SAI (Bẫy đề thi)	Kiến thức ĐÚNG & Cơ chế hoạt động
18	Dùng metric BLEU để chấm bài toán Tóm tắt văn bản	BLEU dùng cho Dịch máy ; Tóm tắt văn bản dùng ROUGE .
19	Áp dụng Data Augmentation trên toàn bộ tập Test	Chỉ augment trên tập Train , tập Validation/Test giữ nguyên.
20	Trong Self-Attention không cần chia cho $\sqrt{d_k}$	Phải chia $\sqrt{d_k}$ để tránh Softmax rơi vào vùng bão hòa gradient.
21	Dùng mô hình BERT để sinh văn bản dài tự do	BERT dùng cho Hiểu (NLU) ; sinh văn bản tự hồi quy dùng GPT .
22	Lớp Pooling (Max, Avg) có tham số huấn luyện	Lớp Pooling hoàn toàn không có tham số học được (0 params) .
23	Thuật toán SMOTE nhân bản y nguyên mẫu thiếu số	SMOTE nội suy sinh điểm mới trên đoạn thẳng nối với láng giềng.
24	$\text{Var}(aX + b) = a \cdot \text{Var}(X)$	Đúng là $\text{Var}(\mathbf{aX} + \mathbf{b}) = \mathbf{a}^2 \cdot \text{Var}(\mathbf{X})$.

§7 Chuyên Đề Tác Vụ Thực Chiến OLP AI 2025 & 2026 (Khung 5 Bước)

Trong các kỳ thi Olympic AI, phần tự luận yêu cầu thí sinh đề xuất phương án giải quyết bài toán thực tế theo **Khung chuẩn 5 bước**:

- Bước 1: Phân tích Dữ liệu & Đặc thù Bài toán** (Kiểu dữ liệu, độ lệch lớp, rủi ro Data Leakage, ràng buộc phần cứng).
- Bước 2: Lựa chọn Mô hình (Baseline → SOTA) & Luận chứng** (Mô hình cơ sở → Mô hình đề xuất chính).
- Bước 3: Pipeline Xử lý & Chiến lược Validation** (Tiền xử lý, Augmentation, chia Fold chống rò rỉ, hậu xử lý).
- Bước 4: Metric Đánh giá Phù hợp** (Chỉ số đo lường phản ánh đúng mục tiêu nghiệp vụ).
- Bước 5: Phương án Cải tiến & Tối ưu Hóa Thực tế** (Kỹ thuật nâng cao: Distillation, Quantization, Ensemble).

§7.1 Tác vụ 1 (Đề OLP AI 2025): Nhận Diện Ngôn Ngữ Ký Hiệu từ Video

Bối cảnh bài toán: 10.000 video clip ngắn ghi lại 50 cử chỉ ngôn ngữ ký hiệu, thực hiện bởi 100 người trong các môi trường ánh sáng và nền phức tạp. Yêu cầu chạy real-time trên laptop.

- Phân tích dữ liệu:** Video có số frame biến thiên theo thời gian. Rủi ro **Data Leakage nghiêm trọng** nếu các frame của cùng một người xuất hiện ở cả Train và Val (mô hình sẽ học nhận diện khuôn mặt người thay vì cử chỉ tay). Cần xử lý bất biến nền.
- Lựa chọn mô hình:**
 - Baseline:* Trích xuất đặc trưng từng frame bằng MobileNetV3 + trượt cửa sổ thời gian.
 - Đề xuất chính:* Kết hợp trích xuất đặc trưng không gian nhẹ (ConvNeXt-Tiny hoặc MediaPipe Holistic trích xuất 3D Landmark bàn tay/khớp xương) kết hợp **Temporal Transformer Encoder / Bi-GRU + Attention Pooling**.
- Pipeline xử lý:** Lấy mẫu cố định 32 frames/clip bằng uniform sampling. Augmentation không gian (Random crop, color jitter) và thời gian (Time masking, co giãn tốc độ $\pm 20\%$). Validation: **Person-independent GroupKFold theo ID người thực hiện**. Huấn luyện bằng hàm mất mát CTC Loss hoặc Cross-Entropy sau Temporal Pooling.
- Metric đánh giá:** **Sequence-level Accuracy**, Word Error Rate (WER), Character Error Rate (CER). Tuyệt đối không dùng Frame-level Accuracy.
- Phương án cải tiến:** Tận dụng tọa độ Landmark từ MediaPipe để chuyển đổi bài toán từ ảnh pixel sang vector khớp xương, tăng tốc độ xử lý 10×; Lượng tử hóa INT8 bằng ONNX Runtime để đạt 30+ FPS trên CPU laptop.

§7.2 Tác vụ 2 (Đề OLP AI 2025): Dịch Máy Thương Mại Điện Tử Hoa — Việt

Bối cảnh bài toán: 200.000 cặp câu song ngữ chuyên ngành thương mại điện tử. Yêu cầu dịch chính xác tên sản phẩm và tuyệt đối không làm sai lệch số lượng, đơn vị tính và mệnh giá tiền tệ.

- Phân tích dữ liệu:** Cặp câu song ngữ bất đối xứng: Tiếng Trung viết liền không khoảng trắng; Tiếng Việt nhiều từ ghép đa âm tiết. Chứa nhiều mã model sản phẩm, giá tiền (¥, VND), số đo (*cm*, *kg*).
- Lựa chọn mô hình:**
 - *Baseline:* Seq2Seq GRU 2 lớp với Luong Attention (huấn luyện từ đầu / Scratch).
 - *Đề xuất chuẩn theo quy chế SOLOAI: Transformer tiêu chuẩn (6 Encoder layers, 6 Decoder layers)* huấn luyện từ đầu (From Scratch) với Joint BPE Tokenizer. *Lưu ý: Thể lệ SOLOAI quy định không được sử dụng mô hình dịch máy Hoa — Việt đã huấn luyện sẵn.*
 - *Phương án mở rộng thực tế (khi thể lệ cho phép):* Fine-tune từ mô hình dịch máy đa ngữ nền tảng như **NLLB-200 (No Language Left Behind)** hoặc **mBART-50**.
- Pipeline xử lý:** Tách từ chuyên biệt bằng Jieba (tiếng Trung) và VnCoreNLP (tiếng Việt). Huấn luyện Joint Byte-Pair Encoding (BPE) chung với vocab 32.000 tokens. **Bảo vệ thực thể số/tiền tệ:** Dùng Regex gán nhãn placeholder (ví dụ <NUM_1>, <CURR_VND>) trước khi dịch, sau đó đối chiếu ánh xạ ngược lại vào câu dịch tiếng Việt. Giải mã bằng Beam Search (Width = 5) kết hợp Label Smoothing = 0.1.
- Metric đánh giá:** **SacreBLEU** (chuẩn hóa tokenization) và **chrF++** (độ đo ký tự nhạy với ngữ pháp tiếng Việt). Đánh giá thủ công trên tập 200 câu khó chứa nhiều thông số kỹ thuật.
- Phương án cải tiến:** Back-translation trên dữ liệu đơn ngữ tiếng Việt để nhân đôi tập train; Reranking các kết quả sinh từ Beam Search bằng mô hình PhoBERT.

§7.3 Tác vụ 3 (Đề xuất OLP 2026): Phát Hiện Bệnh Lá Cây Trồng Trên Thiết Bị Di Động

Bối cảnh bài toán: 8.000 ảnh chụp lá cây ngoài đồng ruộng thực tế, 4 nhóm bệnh (trong đó 2 nhóm bệnh hiếm). Yêu cầu khoanh vùng vết bệnh và chạy mượt mà offline trên điện thoại nông dân.

- Phân tích dữ liệu:** Bài toán Object Detection thực địa với phong nền đất ruộng và bóng râm rất phức tạp. Mất cân bằng lớp nặng giữa bệnh phổ biến và bệnh hiếm. Ràng buộc phần cứng: Dung lượng mô hình < 20 MB, độ trễ < 50 ms.
- Lựa chọn mô hình:**
 - *Baseline:* EfficientNet-B0 làm bộ phân loại toàn ảnh (không khoanh vùng).
 - *Đề xuất chính: YOLOv8n (Nano) hoặc YOLOv11n.* Kiến trúc 1-stage với số tham số chỉ ≈ 3 triệu, tối ưu hóa cao cho bộ xử lý di động NPU/GPU.
- Pipeline xử lý:** Augmentation: Mosaic, Mixup, Random HSV. Validation: **GroupKFold theo thửa ruộng và ngày chụp ảnh.** Áp dụng **Focal Loss** để giảm ảnh hưởng của vùng nền dễ phát hiện, tập trung vào các vết bệnh nhỏ và hiếm. Ngưỡng NMS IoU ≥ 0.5 .
- Metric đánh giá:** mAP@0.5 và mAP@[0.5:0.95]. Báo cáo riêng Precision và Recall cho 2 nhóm bệnh hiếm.
- Phương án cải tiến:** Lượng tử hóa sau huấn luyện (Post-Training Quantization — INT8) xuất sang TFLite/CoreML giúp giảm $4\times$ dung lượng và tăng tốc $3\times$ trên CPU điện thoại.

§7.4 Tác vụ 4 (Đề xuất OLP 2026): Dự Đoán Sinh Viên Bỏ Học & Explainable AI (XAI)

Bối cảnh bài toán: Dữ liệu dạng bảng 50.000 sinh viên với 40 trường thông tin (điểm học phần, số tín chỉ trễ, tình trạng đóng học phí, điểm rèn luyện). Tỉ lệ bỏ học là 8%. Phòng đào tạo cần mô hình dự đoán chính xác và **giải thích được nguyên nhân cụ thể cho từng sinh viên** để can thiệp kịp thời.

- Phân tích dữ liệu:** Dữ liệu dạng bảng (Tabular Data) với 8% nhãn dương (Imbalanced Data). Chứa cả biến số và biến danh mục. Nghiệp vụ đòi hỏi tính minh bạch và khả năng giải thích (Explainability).
- Lựa chọn mô hình:**

- *Baseline*: Logistic Regression sau khi chuẩn hóa StandardScaler.
 - *Đề xuất chính*: **LightGBM hoặc XGBoost**. Vượt trội trên dữ liệu dạng bảng, xử lý tự nhiên giá trị khuyết và tương tác phi tuyến, không cần Deep Learning phức tạp.
3. **Pipeline xử lý**: Điền trung vị cho biến số, tạo nhãn **Missing** cho biến danh mục. Feature Engineering: Tạo các đặc trưng vi phân biểu thị xu hướng: $\Delta GPA = GPA_{\text{kỳ này}} - GPA_{\text{kỳ trước}}$, tỉ lệ tín chỉ nợ / tổng tín chỉ. Validation: **Stratified 5-Fold Cross Validation**. Thiết lập `scale_pos_weight = 11.5`. Tối ưu ngưỡng cắt p^* theo F1-score.
4. **Metric đánh giá**: **PR-AUC** và **F1-score**. Ma trận chi phí (Cost Matrix) phạt nặng việc bỏ sót sinh viên bỏ học (FN).
5. **Phương án cải tiến & Giải thích mô hình (XAI)**: Tích hợp **SHAP (SHapley Additive exPlanations)**:
- *Global Feature Importance*: Xác định các yếu tố cốt lõi dẫn đến nguy cơ bỏ học trên toàn trường.
 - *Local Waterfall Plot*: Vẽ đồ thị đóng góp cụ thể của từng chỉ số cho từng sinh viên cụ thể để gửi cho cố vấn học tập lên phương án hỗ trợ.
-

§8 Bộ Đề Luyện Thi Thực Chiến Chuẩn VOAİ 2026 (50 Câu) — Kèm Đáp Án & Giải Thích Chi Tiết

Abstract

Chương này tập hợp trọn vẹn **50 câu hỏi trắc nghiệm của Đề thi thử Vòng loại VOAİ 2026** (Đỗ Đình Luật). Đây là bộ đề được đánh giá bám sát nhất cấu trúc kiến thức và phong cách ra đề của kỳ thi Olympic AI. Toàn bộ 50 câu đều được trình bày kèm **đáp án chính thức, chứng minh/tính toán toán học chi tiết, mục tra cứu lý thuyết tương ứng trong Handbook (§x.y) và các bẫy đề thi cần phòng tránh**, tạo thành công cụ đối chiếu tuyệt đối cho thí sinh khi luyện đề trên Web App.

Câu 1 (Đề thi thử VOAİ 2026)

Đề bài: Cho ma trận A kích thước $n \times n$. Nếu A là ma trận trực giao (orthogonal matrix), phát biểu nào sau đây là ĐÚNG?

- A. $A^T = A^{-1}$
B. Định thức của A luôn bằng 0.
C. A là ma trận suy biến.
D. Các hàng của A phụ thuộc tuyến tính.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: A

Phân tích & Chứng minh chi tiết:

Theo định nghĩa ma trận trực giao trong đại số tuyến tính: $A^T A = A A^T = I_n$. Điều này trực tiếp suy ra ma trận nghịch đảo $A^{-1} = A^T$. Hơn nữa, lấy định thức hai vế: $\det(A^T A) = \det(A)^2 = \det(I) = 1 \implies \det(A) = \pm 1 \neq 0$, do đó A luôn khả nghịch (không bao giờ suy biến), và các hàng/cột của A tạo thành một hệ trực chuẩn độc lập tuyến tính.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §5.1 & §5.2.

Bẫy đề thi cần tránh: Nhầm lẫn giữa trực giao (orthogonal, $\det = \pm 1$) với ma trận suy biến (singular, $\det = 0$).

Câu 2 (Đề thi thử VOAİ 2026)

Đề bài: Trong tối ưu hóa, nếu ma trận Hessian của hàm số tại một điểm cực trị là xác định dương (positive definite), điểm đó là:

- A. Điểm cực đại địa phương.
B. Điểm cực tiểu địa phương.
C. Điểm yên ngựa.
D. Điểm không xác định.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: B

Phân tích & Chứng minh chi tiết:

Xét khai triển Taylor bậc 2 của hàm $f(\mathbf{x})$ quanh điểm dừng $\mathbf{x}^*$ (nơi $\nabla f(\mathbf{x}^*) = \mathbf{0}$): $f(\mathbf{x}) \approx f(\mathbf{x}^*) + \frac{1}{2}(\mathbf{x} - \mathbf{x}^*)^T H(\mathbf{x} - \mathbf{x}^*)$. Nếu ma trận Hessian $H = \nabla^2 f(\mathbf{x}^*)$ là xác định dương ($H \succ 0$, tức $\mathbf{u}^T H \mathbf{u} > 0$, $\forall \mathbf{u} \neq \mathbf{0}$, mọi trị riêng $\lambda_i > 0$), hàm số cong lồi lên trên tại mọi hướng $\implies \mathbf{x}^*$ là điểm cực tiểu địa phương (local minimum). (Ngược lại xác định âm $\implies$ cực đại; nửa xác định hoặc đổi dấu $\implies$ điểm yên ngựa).

Căn cứ tra cứu trong Handbook: Tham chiếu mục §5.2.

Bẫy đề thi cần tránh: Nhầm lẫn chiều lồi: Đạo hàm bậc hai > 0 tương ứng cực tiểu (min), không phải cực đại (max).

Câu 3 (Đề thi thử VOA1 2026)

Đề bài: (Khó) Tính đạo hàm của hàm số $f(x) = \ln(1 + e^{-x})$. Kết quả nào sau đây đúng?

- A. $f'(x) = \sigma(x) - 1$ (với σ là hàm sigmoid)
B. $f'(x) = \sigma(x)$
C. $f'(x) = 1 + \sigma(x)$
D. $f'(x) = e^{-x}$

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: A

Phân tích & Chứng minh chi tiết:

Áp dụng quy tắc đạo hàm hàm hợp $(\ln u)' = \frac{u'}{u}$: $f'(x) = \frac{(1+e^{-x})'}{1+e^{-x}} = \frac{-e^{-x}}{1+e^{-x}} = -\frac{1}{e^x+1}$. Mặt khác, hàm sigmoid là $\sigma(x) = \frac{1}{1+e^{-x}} = \frac{e^x}{e^x+1}$. Khi đó: $\sigma(x) - 1 = \frac{e^x}{e^x+1} - 1 = \frac{e^x - (e^x+1)}{e^x+1} = -\frac{1}{e^x+1} = f'(x)$. Đây là đạo hàm của hàm Softplus lật ngược, xuất hiện liên tục trong bài toán Logistic Regression và Binary Cross-Entropy.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §2.2 & §5.2.

Bẫy đề thi cần tránh: Quên dấu trừ khi đạo hàm e^{-x} dẫn đến chọn nhầm B hoặc C.

Câu 4 (Đề thi thử VOA1 2026)

Đề bài: Đại lượng nào đo lường mức độ 'bất ngờ' hoặc thông tin trung bình của một phân phối xác suất?

- A. Phương sai.
B. Entropy.
C. Độ lệch chuẩn.
D. Kỳ vọng.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: B

Phân tích & Chứng minh chi tiết:

Trong lý thuyết thông tin của Claude Shannon, lượng tin cá biệt (surprisal) của biến cố x có xác suất $p(x)$ là $I(x) = -\log_2 p(x)$. Kỳ vọng lượng tin trên toàn bộ phân phối chính là Shannon Entropy: $H(X) = \mathbb{E}[I(X)] = -\sum_x p(x) \log_2 p(x)$. Entropy đo lường độ bất định (uncertainty) và mức độ bất ngờ trung bình.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §1.3.

Bẫy đề thi cần tránh: Phương sai chỉ đo độ phân tán của biến ngẫu nhiên số thực quanh giá trị trung bình, không đo thông tin của phân phối xác suất.

Câu 5 (Đề thi thử VOA1 2026)

Đề bài: Trong thuật toán K-Nearest Neighbors (KNN), khi giá trị k quá nhỏ, mô hình có xu hướng:

- A. Bị Underfitting.
B. Bị Overfitting và nhạy cảm với nhiễu.
C. Có độ chệch (Bias) cao.
D. Luôn cho kết quả chính xác nhất.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: B**Phân tích & Chứng minh chi tiết:**

Khi k nhỏ (ví dụ $k = 1$), nhân dự đoán hoàn toàn phụ thuộc vào điểm gần nhất. Ranh giới quyết định (Voronoi) sẽ uốn lượn ôm sát từng điểm dữ liệu, kể cả các điểm nhiễu ngoại lai (outliers) $\implies$ High Variance, Low Bias, dẫn đến hiện tượng Overfitting nghiêm trọng. Ngược lại khi $k \rightarrow N$, mô hình chỉ dự đoán theo lớp đa số $\implies$ Underfitting, High Bias.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §1.1 & §1.5.

Bẫy đề thi cần tránh: Nghĩ rằng $k = 1$ khớp 100% tập train nghĩa là 'kết quả chính xác nhất'.

Câu 6 (Đề thi thử VOA1 2026)

Đề bài: Thuật toán nào sau đây KHÔNG thuộc nhóm học có giám sát (Supervised Learning)?

- A. Support Vector Machine.
- B. Random Forest.
- C. Principal Component Analysis (PCA).
- D. Logistic Regression.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: C**Phân tích & Chứng minh chi tiết:**

PCA (Principal Component Analysis - Phân tích thành phần chính) là thuật toán học không giám sát (Unsupervised Learning) dùng để giảm chiều dữ liệu. PCA tìm các trục trực giao cực đại hóa phương sai của tập dữ liệu mà hoàn toàn không sử dụng bất kỳ nhãn mục tiêu y nào.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §1.1 & §1.8.

Bẫy đề thi cần tránh: Nhầm Logistic Regression là Unsupervised vì chữ 'Regression'.

Câu 7 (Đề thi thử VOA1 2026)

Đề bài: Mục tiêu chính của kỹ thuật 'Pruning' (tỉa cành) trong cây quyết định là gì?

- A. Tăng độ sâu của cây.
- B. Giảm hiện tượng Overfitting.
- C. Tăng số lượng nút lá.
- D. Làm cho cây phức tạp hơn.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: B**Phân tích & Chứng minh chi tiết:**

Cây quyết định nếu phát triển không giới hạn sẽ tiếp tục chia nhánh tới khi mọi nút lá đều thuần khiết, học cả nhiễu và overfit. Kỹ thuật tỉa cành (Pre-pruning: giới hạn max_depth, min_samples_split; hoặc Post-pruning: Cost-Complexity Pruning) loại bỏ các nhánh cây con không mang lại ý nghĩa thống kê trên tập validation, giúp giảm phương sai và kiểm soát Overfitting.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §1.3.

Bẫy đề thi cần tránh: Nhầm tỉa cành là làm tăng độ sâu để nâng cao accuracy trên tập train.

Câu 8 (Đề thi thử VOA1 2026)

Đề bài: Thuật toán Random Forest kết hợp nhiều cây quyết định bằng phương pháp:

- A. Boosting.
- B. Bagging.
- C. Stacking.
- D. Cascading.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: B

Phân tích & Chứng minh chi tiết:

Random Forest là phương pháp Bagging (Bootstrap Aggregating) kết hợp Random Subspaces. Mô hình tạo ra B tập con bằng cách lấy mẫu ngẫu nhiên có hoàn lại (bootstrap) từ tập train gốc, huấn luyện song song B cây quyết định độc lập, và tổng hợp kết quả bằng biểu quyết đa số (classification) hoặc trung bình (regression) nhằm giảm phương sai.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §1.4.

Bẫy đề thi cần tránh: Nhầm giữa Bagging (song song, giảm variance) và Boosting (tuần tự, giảm bias).

Câu 9 (Đề thi thử VOA1 2026)

Đề bài: Chỉ số F1-Score là trung bình điều hòa của hai đại lượng nào?

- A. Accuracy và Recall.
- B. Precision và Accuracy.
- C. Precision và Recall.
- D. Specificity và Sensitivity.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: C

Phân tích & Chứng minh chi tiết:

Chỉ số $F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$. Trung bình điều hòa (Harmonic Mean) được sử dụng vì nó phạt rất nặng khi một trong hai đại lượng tiệm cận 0, đảm bảo mô hình cân bằng tốt giữa khả năng phân loại chính xác mẫu dương (Precision) và khả năng bắt trọn mẫu dương (Recall).

Căn cứ tra cứu trong Handbook: Tham chiếu mục §1.7.

Bẫy đề thi cần tránh: Nhầm $F1$ là trung bình cộng (Arithmetic Mean) giữa Precision và Recall.

Câu 10 (Đề thi thử VOA1 2026)

Đề bài: Trong Linear Regression, nếu các biến độc lập có tương quan rất mạnh với nhau, hiện tượng này gọi là:

- A. Đa cộng tuyến (Multicollinearity).
- B. Tự tương quan.
- C. Phương sai thay đổi.
- D. Nhiễu trắng.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: A**Phân tích & Chứng minh chi tiết:**

Hiện tượng đa cộng tuyến (Multicollinearity) xuất hiện khi hai hoặc nhiều biến giải thích X_i, X_j có tương quan tuyến tính cao. Khi đó ma trận hiệp phương sai $X^T X$ có định thức gần bằng 0 (gần suy biến), ma trận nghịch đảo $(X^T X)^{-1}$ không ổn định số học, khiến phương sai của các hệ số hồi quy β bùng nổ, làm dấu của trọng số bị đảo lộn bất thường.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §1.6 & §5.3.

Bẫy đề thi cần tránh: Tự tương quan (Autocorrelation) là tương quan của chính một biến với các bước trễ của nó trong dữ liệu chuỗi thời gian, không phải giữa các biến khác nhau.

Câu 11 (Đề thi thử VOA1 2026)

Đề bài: Hàm loss nào thường được dùng cho bài toán Logistic Regression?

- A. Mean Squared Error.
- B. Binary Cross-Entropy (Log Loss).
- C. Hinge Loss.
- D. Huber Loss.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: B**Phân tích & Chứng minh chi tiết:**

Logistic Regression mô hình hóa xác suất bằng hàm Sigmoid $p = \sigma(\mathbf{w}^T \mathbf{x})$. Tối ưu hóa ước lượng hợp lý cực đại (MLE) dẫn trực tiếp đến hàm mất mát Binary Cross-Entropy (Log Loss): $\mathcal{L} = -\frac{1}{N} \sum [y_i \ln p_i + (1 - y_i) \ln(1 - p_i)]$. MSE nếu dùng cho Logistic Regression sẽ tạo ra hàm mất mát không lồi (non-convex) có nhiều cực tiểu địa phương giả.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §1.1 & §2.4.

Bẫy đề thi cần tránh: Nghĩ rằng dùng MSE cho Logistic Regression vẫn được (thực tế MSE tạo loss surface không lồi).

Câu 12 (Đề thi thử VOA1 2026)

Đề bài: (Khó) Cho ma trận nhầm lẫn: TP=80, FP=10, FN=20, TN=90. Precision của mô hình là:

- A. 0.8
- B. 0.88
- C. 0.75
- D. 0.9

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: B**Phân tích & Chứng minh chi tiết:**

Công thức Precision: $\text{Precision} = \frac{TP}{TP+FP} = \frac{80}{80+10} = \frac{80}{90} \approx 0.8888... \approx 0.88$ (hoặc 0.89). Trong khi đó Recall = $\frac{TP}{TP+FN} = \frac{80}{80+20} = 0.80$; Accuracy = $\frac{TP+TN}{\text{Total}} = \frac{80+90}{200} = 0.85$.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §1.7.

Bẫy đề thi cần tránh: Chia nhầm mẫu số thành $TP + FN$ (ra 0.80 là Recall) hoặc $TP + TN$.

Câu 13 (Đề thi thử VOA1 2026)

Đề bài: Tại sao hàm kích hoạt phi tuyến là cần thiết trong mạng nơ-ron?

- A. Để làm mạng chạy nhanh hơn.
- B. Để mạng có thể học được các ranh giới quyết định phức tạp (phi tuyến).
- C. Để giới hạn giá trị trọng số.
- D. Để thay thế lớp Dropout.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: B

Phân tích & Chứng minh chi tiết:

Nếu các hàm kích hoạt đều là tuyến tính $f(z) = cz$, việc ghép nối nhiều lớp ẩn liên tiếp chỉ tương đương với một phép nhân ma trận đơn lẻ duy nhất: $\mathbf{W}_2(\mathbf{W}_1\mathbf{x}) = (\mathbf{W}_2\mathbf{W}_1)\mathbf{x} = \mathbf{W}_{net}\mathbf{x}$, biến toàn bộ mạng sâu thành một mô hình tuyến tính đơn giản (không thể học bài toán XOR). Hàm phi tuyến phá vỡ tính chất tuyến tính, cho phép mạng xấp xỉ bất kỳ hàm số liên tục nào theo Định lý Xấp xỉ Phổ quát (Universal Approximation Theorem).

Căn cứ tra cứu trong Handbook: Tham chiếu mục §2.1 & §2.2.

Bẫy đề thi cần tránh: Nghĩ rằng phi tuyến giúp mạng chạy nhanh hơn (thực tế hàm phi tuyến tốn tài nguyên tính toán hơn).

Câu 14 (Đề thi thử VOA1 2026)

Đề bài: (Khó) Một lớp Convolution có filter 5×5 , input channels là 3, số lượng filter là 16. Tổng số tham số (không tính bias) là:

- A. 1200
- B. 400
- C. 240
- D. 384

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: A

Phân tích & Chứng minh chi tiết:

Mỗi bộ lọc (filter) phải quét qua toàn bộ các kênh của đầu vào ($C_{in} = 3$), do đó kích thước thực sự của một kernel 3D là $K_h \times K_w \times C_{in} = 5 \times 5 \times 3 = 75$ trọng số. Với 16 filter riêng biệt để sinh ra 16 kênh đầu ra ($C_{out} = 16$), tổng số tham số trọng số là: $75 \times 16 = 1200$. (Nếu tính cả hệ số bias thì cộng thêm 16 bias $\implies 1216$).

Căn cứ tra cứu trong Handbook: Tham chiếu mục §3.1.

Bẫy đề thi cần tránh: Quên nhân với số kênh đầu vào ($C_{in} = 3$): $5 \times 5 \times 16 = 400$ (sai thiếu kênh).

Câu 15 (Đề thi thử VOA1 2026)

Đề bài: Trong LSTM, cổng (gate) nào quyết định thông tin nào từ trạng thái ô (cell state) cũ sẽ bị loại bỏ?

- A. Input Gate.
- B. Forget Gate.
- C. Output Gate.
- D. Update Gate.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: B**Phân tích & Chứng minh chi tiết:**

Cổng quên (Forget Gate) trong LSTM tính toán: $\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \in (0, 1)$. Giá trị của $\mathbf{f}_t$ được nhân từng phần tử với trạng thái ô trước đó $\mathbf{C}_{t-1}$. Giá trị gần 0 đồng nghĩa 'quên hoàn toàn', giá trị gần 1 đồng nghĩa 'giữ nguyên hoàn toàn'.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §4.4.

Bẫy đề thi cần tránh: Nhầm Input Gate là cổng quyết định xóa thông tin (Input Gate quyết định nạp thông tin mới).

Câu 16 (Đề thi thử VOA1 2026)

Đề bài: (Khó) Cho đầu vào $W \times H$, filter $K \times K$, padding P , stride S . Công thức tính kích thước đầu ra W_{out} là:

- A. $(W - K + 2P)/S + 1$ (làm tròn xuống $\lfloor \cdot \rfloor$)
- B. $(W + K - P)/S$
- C. $W/S + P$
- D. $(W - K + P)/S$

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: A**Phân tích & Chứng minh chi tiết:**

Công thức kinh điển xác định kích thước không gian đầu ra của tầng Conv2D: $W_{out} = \lfloor \frac{W-K+2P}{S} \rfloor + 1$. Padding P được thêm vào cả hai phía (trái và phải) nên tổng độ rộng được bù là $2P$.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §3.1.

Bẫy đề thi cần tránh: Chỉ cộng P thay vì $2P$ (quên padding ở cả hai đầu biên).

Câu 17 (Đề thi thử VOA1 2026)

Đề bài: Hàm kích hoạt nào thường được dùng ở lớp cuối của bài toán phân loại đa lớp (Multi-class Classification)?

- A. Sigmoid.
- B. Softmax.
- C. Tanh.
- D. ReLU.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: B**Phân tích & Chứng minh chi tiết:**

Hàm Softmax biến đổi vector logit $\mathbf{z} \in \mathbb{R}^C$ thành phân phối xác suất hợp lệ: $\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}$, thỏa mãn $\sum_{i=1}^C \sigma(\mathbf{z})_i = 1$ và $\sigma(\mathbf{z})_i \in (0, 1)$, phù hợp với phân loại đa lớp loại trừ lẫn nhau (mutually exclusive). Sigmoid chỉ dùng cho phân loại nhị phân hoặc đa nhãn (multi-label).

Căn cứ tra cứu trong Handbook: Tham chiếu mục §2.2 & §2.4.

Bẫy đề thi cần tránh: Dùng Sigmoid cho đa lớp (Sigmoid cho tổng xác suất > 1 , chỉ đúng với bài toán Multi-label).

Câu 18 (Đề thi thử VOA1 2026)

Đề bài: Trong thuật toán tối ưu Adam, tham số β_1 và β_2 thường được dùng để:

- A. Tính toán trung bình động của gradient và bình phương gradient.
- B. Thay đổi kích thước batch.
- C. Khởi tạo trọng số.
- D. Cắt (clip) gradient.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: A

Phân tích & Chứng minh chi tiết:

Adam duy trì hai moment: β_1 (mặc định 0.9) là hệ số suy giảm của trung bình động gradient (Moment bậc 1: $m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$), tương tự Momentum. β_2 (mặc định 0.999) là hệ số của trung bình động bình phương gradient (Moment bậc 2: $v_t = \beta_2 v_{t-1} + (1 - \beta_2)g_t^2$), tương tự RMSProp.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §2.5.

Bẫy đề thi cần tránh: Nhầm β_1, β_2 với hệ số clipping gradient hay learning rate.

Câu 19 (Đề thi thử VOA1 2026)

Đề bài: Mạng Generative Adversarial Networks (GAN) bao gồm hai thành phần chính nào?

- A. Encoder và Decoder.
- B. Generator và Discriminator.
- C. Actor và Critic.
- D. Input và Output.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: B

Phân tích & Chứng minh chi tiết:

GAN (Goodfellow et al., 2014) gồm Mạng sinh (Generator G) cố gắng biến vector nhiễu ngẫu nhiên thành mẫu giả giống thật, và Mạng phân định (Discriminator D) cố gắng phân biệt mẫu thật từ dataset và mẫu giả từ G . Hai mạng được huấn luyện đối kháng qua trò chơi minimax hai người.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §3.8.

Bẫy đề thi cần tránh: Encoder và Decoder là kiến trúc của VAE (Variational Autoencoder) hoặc Transformer, không phải GAN.

Câu 20 (Đề thi thử VOA1 2026)

Đề bài: Một nơ-ron có 3 đầu vào (1, 2, 3), trọng số tương ứng là (0.5, -1, 2) và bias là 0.5. Nếu dùng hàm ReLU, đầu ra là:

- A. 5
- B. 0
- C. -4.5
- D. 4.5

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: A**Phân tích & Chứng minh chi tiết:**

Tổng tuyến tính đầu vào: $z = \sum w_i x_i + b = (1 \times 0.5) + (2 \times -1) + (3 \times 2) + 0.5 = 0.5 - 2.0 + 6.0 + 0.5 = 5.0$.

Áp dụng hàm kích hoạt ReLU: $\text{ReLU}(z) = \max(0, z) = \max(0, 5.0) = 5.0$.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §2.1.

Bẫy đề thi cần tránh: *Quên cộng bias 0.5 dẫn đến ra 4.5 (chọn D).*

Câu 21 (Đề thi thử VOA1 2026)

Đề bài: Phương pháp 'Bag of Words' (BoW) có nhược điểm lớn nhất là gì?

- A. Mất hoàn toàn thông tin về thứ tự và ngữ cảnh của từ.
- B. Tính toán quá chậm.
- C. Không thể đếm được tần suất từ.
- D. Chỉ dùng được cho tiếng Anh.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: A**Phân tích & Chứng minh chi tiết:**

BoW chỉ biểu diễn văn bản dưới dạng vector đếm tần số từ độc lập, bỏ qua hoàn toàn trật tự từ (Word Order) và ngữ pháp cấu trúc câu. Ví dụ: 'Tôi không thích anh ta nhưng tôi thích bạn' và 'Tôi thích anh ta nhưng tôi không thích bạn' sẽ có biểu diễn BoW hoàn toàn giống nhau.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §4.2.

Bẫy đề thi cần tránh: *Nghĩ rằng BoW tính toán chậm (thực tế BoW tính rất nhanh nhưng biểu diễn thô).*

Câu 22 (Đề thi thử VOA1 2026)

Đề bài: 'BERT' là mô hình dựa trên thành phần nào của Transformer?

- A. Encoder.
- B. Decoder.
- C. Cả hai.
- D. Không thành phần nào.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: A**Phân tích & Chứng minh chi tiết:**

BERT (Bidirectional Encoder Representations from Transformers) được xây dựng hoàn toàn từ chồng các khối Transformer Encoder, sử dụng cơ chế Self-Attention hai chiều (Bidirectional) để hiểu sâu sắc ngữ cảnh hai phía. Dòng GPT sử dụng Transformer Decoder (Autoregressive), còn T5/BART sử dụng cả Encoder lẫn Decoder.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §4.6.

Bẫy đề thi cần tránh: *Nhầm BERT với GPT (Decoder) hoặc mô hình dịch máy gốc (gồm cả hai).*

Câu 23 (Đề thi thử VOA1 2026)

Đề bài: Hệ thống RAG (Retrieval-Augmented Generation) giúp mô hình LLM:

- A. Giảm hiện tượng 'ảo giác' (hallucination) bằng cách tham chiếu dữ liệu bên ngoài.
- B. Chạy nhanh hơn 10 lần.
- C. Không cần huấn luyện lại.
- D. Cả A và C đều đúng.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: D

Phân tích & Chứng minh chi tiết:

RAG kết hợp một hệ thống truy xuất (Retriever) và một mô hình sinh (Generator). Khi người dùng hỏi, tài liệu liên quan từ kho dữ liệu ngoài được truy xuất và đính kèm vào ngữ cảnh của LLM. Kỹ thuật này vừa giảm ảo giác (hallucination) do có bằng chứng thực tế, vừa cho phép cập nhật tri thức mới tức thì mà không cần tốn chi phí train lại mô hình.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §4.6 & §4.8.

Bẫy đề thi cần tránh: Chỉ chọn A mà quên mất C (hoặc ngược lại).

Câu 24 (Đề thi thử VOA1 2026)

Đề bài: Phép toán 'Max Pooling' có tác dụng:

- A. Giảm kích thước không gian và giữ lại đặc trưng nổi bật nhất.
- B. Tăng số lượng kênh.
- C. Thay đổi màu sắc ảnh.
- D. Tăng độ phân giải.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: A

Phân tích & Chứng minh chi tiết:

Max Pooling trượt một cửa sổ (ví dụ 2×2 , stride 2) trên từng feature map và lấy giá trị lớn nhất trong cửa sổ. Nó làm giảm 1/2 kích thước không gian chiều rộng và chiều cao (giảm 75% số điểm ảnh), giảm số phép tính của các tầng sau, tạo tính bất biến dịch chuyển nhẹ (translation invariance), và giữ lại tín hiệu kích hoạt mạnh nhất.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §3.2.

Bẫy đề thi cần tránh: Nghĩ rằng Pooling tăng số lượng kênh (Pooling giữ nguyên số kênh C).

Câu 25 (Đề thi thử VOA1 2026)

Đề bài: Thuật toán 'YOLO' thuộc nhóm phát hiện đối tượng:

- A. Single-stage detector.
- B. Two-stage detector.
- C. Unsupervised detector.
- D. Phân đoạn cá thể (Instance Segmentation).

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: A**Phân tích & Chứng minh chi tiết:**

YOLO (You Only Look Once) thuộc nhóm Single-stage Detector. Mô hình chia ảnh thành lưới và dự đoán đồng thời tọa độ bounding box cùng xác suất nhân lớp trong một lần lan truyền tiến (single forward pass) duy nhất, đạt tốc độ thời gian thực (Real-time). Dòng R-CNN (Faster R-CNN) thuộc nhóm Two-stage Detector.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §3.6.

Bẫy đề thi cần tránh: Nhầm YOLO với Faster R-CNN (Two-stage detector có RPN).

Câu 26 (Đề thi thử VOA1 2026)

Đề bài: Thuật toán NMS (Non-Maximum Suppression) dùng để:

- A. Loại bỏ các khung hình (bounding boxes) trùng lặp và chỉ giữ lại khung có xác suất cao nhất.
- B. Tăng số lượng dự đoán.
- C. Làm mượt ảnh.
- D. Tính toán hàm loss.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: A**Phân tích & Chứng minh chi tiết:**

Trong Object Detection, mạng thường tạo ra hàng chục box dự đoán chồng chéo lên cùng một đối tượng. NMS sắp xếp các box theo điểm tin cậy giảm dần, chọn box cao nhất, sau đó tính IoU với các box lân cận; nếu $\text{IoU} > \text{threshold}$ (thường 0.45 – 0.5) thì loại bỏ các box trùng lặp đó.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §3.6.

Bẫy đề thi cần tránh: Nhầm NMS là thuật toán làm mượt ảnh hoặc hàm tối ưu.

Câu 27 (Đề thi thử VOA1 2026)

Đề bài: 'Model Quantization' (Lượng hóa mô hình) giúp:

- A. Giảm dung lượng mô hình và tăng tốc độ suy luận (inference) trên thiết bị nhúng.
- B. Tăng độ chính xác của mô hình.
- C. Thêm nhiều nơ-ron hơn.
- D. Chuyển mô hình từ Python sang Java.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: A**Phân tích & Chứng minh chi tiết:**

Quantization chuyển đổi biểu diễn số học của trọng số và kích hoạt từ dấu phẩy động độ chính xác cao (FP32 hoặc FP16) sang định dạng bit thấp hơn (INT8, INT4, NF4). Điều này giúp giảm 50%–75% dung lượng RAM/VRAM, giảm băng thông bộ nhớ và tăng tốc độ suy luận lên đáng kể trên chip di động/edge.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §2.7 & §7.3.

Bẫy đề thi cần tránh: Nghĩ rằng lượng hóa làm tăng độ chính xác (thực tế lượng hóa có thể làm giảm nhẹ độ chính xác).

Câu 28 (Đề thi thử VOA1 2026)

Đề bài: Khái niệm 'Data Drift' (Trôi dạt dữ liệu) nghĩa là:

- A. Sự thay đổi phân phối của dữ liệu thực tế theo thời gian khiến mô hình cũ không còn hiệu quả.
- B. Dữ liệu bị xóa mất.
- C. Dữ liệu bị sao chép.
- D. Tốc độ truyền dữ liệu chậm.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: A

Phân tích & Chứng minh chi tiết:

Data Drift (Covariate Shift) xảy ra khi phân phối dữ liệu đầu vào trong môi trường thực tế thay đổi theo thời gian so với phân phối tập huấn luyện: $P_{production}(X) \neq P_{training}(X)$, ví dụ hành vi người dùng thay đổi theo mùa hoặc cảm biến bị lão hóa, dẫn đến hiệu năng của mô hình bị thoái hóa.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §5.6 & §7.4.

Bẫy đề thi cần tránh: Nhầm Data Drift với lỗi mất mát dữ liệu phân cứng.

Câu 29 (Đề thi thử VOA1 2026)

Đề bài: 'MLOps' là viết tắt của:

- A. Machine Learning Operations.
- B. Multi-Layer Optimization.
- C. Mobile Learning Options.
- D. Main Logic Operator.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: A

Phân tích & Chứng minh chi tiết:

MLOps (Machine Learning Operations) là sự giao thoa giữa Machine Learning, DevOps và Data Engineering nhằm chuẩn hóa toàn bộ vòng đời phát triển: CI/CD/CT (Continuous Training), tự động hóa triển khai, quản lý phiên bản dữ liệu/mô hình (DVC, MLflow) và giám sát drift trong production.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §7.

Bẫy đề thi cần tránh: Các lựa chọn viết tắt giả tạo như Multi-Layer Optimization.

Câu 30 (Đề thi thử VOA1 2026)

Đề bài: Để phục vụ (serving) một mô hình LLM lớn, kỹ thuật nào thường được dùng để tiết kiệm VRAM?

- A. Paging.
- B. KV Caching.
- C. Unzipping.
- D. Overclocking.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: B**Phân tích & Chứng minh chi tiết:**

Trong quá trình sinh văn bản tự hồi quy (Autoregressive), mỗi token mới sinh ra cần truy vấn lại toàn bộ ngữ cảnh trước đó. KV Caching lưu lại các tensor Key và Value của các token trước trên VRAM để tránh việc phải tính toán lại từ đầu. Kỹ thuật PagedAttention (vLLM) tổ chức KV Cache thành các trang bộ nhớ liên kết ảo để chống phân mảnh và tiết kiệm tối đa VRAM.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §2.7 & §4.5.

Bẫy đề thi cần tránh: Nhầm với các thuật ngữ ép xung phần cứng (Overclocking) hoặc giải nén.

Câu 31 (Đề thi thử VOA1 2026)

Đề bài: Học bán giám sát (Semi-supervised learning) thường được áp dụng hiệu quả nhất trong kịch bản nào?

- A. Toàn bộ dữ liệu đều đã được gán nhãn rất chuẩn xác.
- B. Không có bất kỳ dữ liệu nào được gán nhãn.
- C. Có một lượng nhỏ dữ liệu được gán nhãn và một lượng lớn dữ liệu chưa gán nhãn.
- D. Số lượng nhãn cần phân loại thay đổi liên tục theo thời gian.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: C**Phân tích & Chứng minh chi tiết:**

Trong thực tế, việc thu thập dữ liệu thô thường rất rẻ nhưng việc gán nhãn thủ công (bởi chuyên gia y tế, kỹ sư) lại cực kỳ đắt đỏ. Semi-supervised learning sử dụng tập nhỏ mẫu có nhãn để huấn luyện mô hình ban đầu, sau đó kết hợp cấu trúc phân phối của tập lớn dữ liệu không nhãn (qua Pseudo-labeling, Consistency Regularization) để cải thiện độ khái quát hóa.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §1.1.

Bẫy đề thi cần tránh: Nhầm với Unsupervised (hoàn toàn không có nhãn) hoặc Supervised (100% có nhãn).

Câu 32 (Đề thi thử VOA1 2026)

Đề bài: Điểm vượt trội của thuật toán t-SNE so với PCA khi giảm chiều dữ liệu trực quan hóa là gì?

- A. t-SNE có tốc độ tính toán nhanh hơn rất nhiều trên tập dữ liệu lớn.
- B. t-SNE bảo toàn tốt hơn cấu trúc phi tuyến và khoảng cách cục bộ (local structure).
- C. t-SNE luôn giữ lại 100% phương sai của dữ liệu gốc.
- D. t-SNE sử dụng phép chiếu tuyến tính đơn giản nên dễ suy luận ngược.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: B**Phân tích & Chứng minh chi tiết:**

PCA là phép chiếu tuyến tính toàn cục, thường làm các cụm dữ liệu phi tuyến phức tạp đè bẹp lên nhau khi chiếu xuống 2D. t-SNE chuyển đổi khoảng cách Euclid giữa các điểm thành xác suất có điều kiện và sử dụng phân phối Student-t ở không gian thấp chiều, giúp bảo tồn xuất sắc các mối quan hệ láng giềng phi tuyến cục bộ (local manifold).

Căn cứ tra cứu trong Handbook: Tham chiếu mục §1.8.

Bẫy đề thi cần tránh: Nghĩ rằng t-SNE chạy nhanh hơn PCA (thực tế t-SNE chậm hơn PCA rất nhiều, độ phức tạp $O(N^2)$).

Câu 33 (Đề thi thử VOA1 2026)

Đề bài: Tại sao Stratified K-Fold lại được ưu tiên sử dụng hơn K-Fold thông thường trong các bài toán phân loại có dữ liệu mất cân bằng?

- A. Nó đảm bảo tỷ lệ giữa các lớp trong mỗi Fold giống với tỷ lệ trong toàn bộ tập dữ liệu.
- B. Nó giúp mô hình chạy nhanh hơn bằng cách giảm bớt số lượng mẫu.
- C. Nó tự động loại bỏ các dữ liệu ngoại lệ (outliers) gây nhiễu.
- D. Nó không cho phép các Fold có dữ liệu trùng lặp với nhau.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: A

Phân tích & Chứng minh chi tiết:

Nếu một bài toán có lớp hiếm chỉ chiếm 1%, K-Fold ngẫu nhiên có thể tạo ra các fold hoàn toàn không có mẫu dương nào trong tập validation. Stratified K-Fold phân tầng dữ liệu sao cho tỷ lệ giữa các lớp trong mỗi fold con đại diện chính xác tỷ lệ phân bố của toàn bộ tập dữ liệu ban đầu, giúp đánh giá metric (F1, AUC) ổn định và khách quan.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §1.5 & §1.9.

Bẫy đề thi cần tránh: Nghĩ rằng Stratified K-Fold làm giảm số lượng mẫu hoặc loại bỏ outlier.

Câu 34 (Đề thi thử VOA1 2026)

Đề bài: Hiện tượng 'Data Leakage' (Rò rỉ dữ liệu) trong quá trình xử lý đặc trưng (Feature Engineering) thường xảy ra khi:

- A. Chúng ta chuẩn hóa dữ liệu (Scaling) trên toàn bộ dataset trước khi chia tập Train/Test.
- B. Tập dữ liệu Train có số lượng mẫu quá nhỏ so với tập Test.
- C. Dùng kỹ thuật K-Fold cross-validation để đánh giá mô hình.
- D. Mô hình có độ phức tạp quá thấp dẫn đến underfitting.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: A

Phân tích & Chứng minh chi tiết:

Nếu gọi `scaler.fit_transform(full_data)` trước khi chia `train_test_split`, các thông tin thống kê của tập test (như mean μ và standard deviation σ) đã bị rò rỉ vào quá trình học của tập train. Khi đó kết quả kiểm thử trên test set sẽ bị lạc quan giả tạo. Quy trình chuẩn: Luôn chia tập trước, chỉ 'fit' trên Train, sau đó áp dụng 'transform' trên Test.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §1.5 & §5.6.

Bẫy đề thi cần tránh: Nhầm Data Leakage là một dạng Underfitting do mô hình đơn giản.

Câu 35 (Đề thi thử VOA1 2026)

Đề bài: Kỹ thuật SMOTE (Synthetic Minority Over-sampling Technique) giải quyết vấn đề Class Imbalance bằng cách:

- A. Loại bỏ ngẫu nhiên các mẫu thuộc lớp đa số (majority class).
- B. Tạo ra các mẫu tổng hợp mới cho lớp thiểu số dựa trên khoảng cách K-NN.
- C. Gán trọng số cao hơn cho các mẫu thuộc lớp đa số trong hàm loss.
- D. Nhân bản lặp đi lặp lại các mẫu có sẵn của lớp thiểu số.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: B**Phân tích & Chứng minh chi tiết:**

Khác với Random Over-sampling (nhân bản trùng lặp các điểm có sẵn dễ dẫn tới overfit), SMOTE chọn một điểm mẫu lớp thiểu số x_i , tìm k láng giềng gần nhất cùng lớp của nó, chọn ngẫu nhiên một láng giềng x_{zi} , và sinh điểm mới bằng nội suy tuyến tính: $x_{new} = x_i + \lambda(x_{zi} - x_i)$ với $\lambda \in [0, 1]$.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §1.9.

Bẫy đề thi cần tránh: Chọn D (nhân bản lặp lại là Random Over-sampling, không phải SMOTE).

Câu 36 (Đề thi thử VOA1 2026)

Đề bài: Khi đánh giá một mô hình phân loại trên tập dữ liệu cực kỳ mất cân bằng (như phát hiện gian lận thẻ tín dụng), độ đo nào sau đây thường bị coi là 'vô dụng' và gây hiểu lầm nhất?

- A. F1-score.
- B. Precision-Recall AUC.
- C. Accuracy (Độ chính xác tổng thể).
- D. Recall.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: C**Phân tích & Chứng minh chi tiết:**

Giả sử trong 10,000 giao dịch chỉ có 10 vụ gian lận (tỷ lệ 0.1%). Một mô hình hoàn toàn không học gì, luôn dự đoán mọi giao dịch là 'hợp lệ' (negative) thì Accuracy vẫn đạt $9990/10000 = 99.9\%$. Tuy nhiên mô hình này hoàn toàn vô dụng vì phát hiện được 0% vụ gian lận. Do đó trên dữ liệu lệch, Accuracy là độ đo gây ngộ nhận nghiêm trọng nhất.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §1.7 & §1.9.

Bẫy đề thi cần tránh: Thói quen luôn dùng Accuracy cho mọi bài toán phân loại.

Câu 37 (Đề thi thử VOA1 2026)

Đề bài: Điểm khác biệt cơ bản về mặt hiệu ứng giữa Regularization L1 (Lasso) và L2 (Ridge) là gì?

- A. L1 có xu hướng triệt tiêu các trọng số về 0 (tạo tính thưa thớt), trong khi L2 chỉ thu nhỏ chúng.
- B. L2 triệt tiêu hoàn toàn các trọng số về 0, L1 thì không.
- C. L1 chạy nhanh hơn L2 trên mọi kiến trúc phần cứng.
- D. L1 chỉ dùng cho hồi quy tuyến tính, L2 dùng cho mạng nơ-ron.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: A**Phân tích & Chứng minh chi tiết:**

Do hình học của quả cầu chuẩn L1 có các góc nhọn nằm ngay trên các trục tọa độ (hình thoi trong 2D), các đường đồng mức của hàm loss thường tiếp xúc với biên ràng buộc L1 tại chính các đỉnh trục tọa độ $\implies$ một số trọng số bị ép chính xác về 0 (tạo tính thưa thớt - Sparsity, đóng vai trò chọn lọc đặc trưng Feature Selection). Chuẩn L2 (hình cầu tròn) chỉ kéo các trọng số co nhỏ dần về gần 0 nhưng không bao giờ bằng 0 tuyệt đối.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §1.6.

Bẫy đề thi cần tránh: Đảo ngược vai trò: nghĩ rằng L2 triệt tiêu về 0 còn L1 chỉ thu nhỏ.

Câu 38 (Đề thi thử VOA1 2026)

Đề bài: Trong cây quyết định (Decision Tree), việc thiết lập tham số `max_depth` quá lớn mà không có cơ chế cắt tỉa sẽ dẫn đến:

- A. Mô hình bị Underfitting nặng.
- B. Mô hình bị Overfitting do cây quá sâu và học cả nhiễu.
- C. Thời gian suy luận (Inference) giảm đi đáng kể.
- D. Độ chệch (Bias) của mô hình tăng cao.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: B

Phân tích & Chứng minh chi tiết:

Độ sâu của cây quyết định kiểm soát độ phức tạp của mô hình. Khi ‘`max_depth`’ quá lớn, cây sẽ tiếp tục phân nhánh đến tận từng mẫu riêng lẻ, tạo ra các lát cắt siêu nhỏ ghi nhớ cả ngoại lai và nhiễu ngẫu nhiên của tập train, khiến $\text{Training Error} = 0$ nhưng Test Error bùng nổ $\implies$ Overfitting (High Variance).

Căn cứ tra cứu trong Handbook: Tham chiếu mục §1.3.

Bẫy đề thi cần tránh: Nhầm Overfitting với Underfitting.

Câu 39 (Đề thi thử VOA1 2026)

Đề bài: Trong phân tích dữ liệu khám phá (EDA), biểu đồ Boxplot (biểu đồ hộp) là công cụ cực kỳ hữu hiệu để nhanh chóng phát hiện:

- A. Các giá trị ngoại lệ (Outliers) và phân phối tứ phân vị của dữ liệu.
- B. Mọi quan hệ tuyến tính giữa hai biến liên tục.
- C. Tần suất xuất hiện của các từ trong văn bản.
- D. Ma trận tương quan giữa tất cả các đặc trưng.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: A

Phân tích & Chứng minh chi tiết:

Boxplot trực quan hóa ‘Tóm tắt 5 số’ của phân phối: Minimum, Tứ phân vị thứ nhất (Q_1), Trung vị (Q_2), Tứ phân vị thứ ba (Q_3) và Maximum. Khoảng trái giữa $IQR = Q_3 - Q_1$. Các điểm nằm ngoài khoảng râu $[Q_1 - 1.5 \times IQR, Q_3 + 1.5 \times IQR]$ được biểu diễn bằng các chấm riêng biệt, giúp nhận diện Outliers ngay lập tức.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §5.3 & §7.

Bẫy đề thi cần tránh: Nhầm Boxplot với Scatter plot (dùng xem quan hệ tuyến tính giữa 2 biến).

Câu 40 (Đề thi thử VOA1 2026)

Đề bài: Kỹ thuật Early Stopping dừng quá trình huấn luyện mạng nơ-ron dựa trên tín hiệu nào?

- A. Khi Train loss đạt giá trị bằng 0 tuyệt đối.
- B. Khi Validation loss không còn giảm hoặc bắt đầu tăng liên tục trong một số epoch.
- C. Khi số lượng epoch đạt đúng giới hạn do người dùng đặt ra từ đầu.
- D. Khi tốc độ tính toán của GPU bị sụt giảm.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: B**Phân tích & Chứng minh chi tiết:**

Khi mạng nơ-ron bắt đầu học vẹt (overfit), Train loss vẫn tiếp tục giảm nhưng Validation loss sẽ chạm đáy rồi bắt đầu tăng lên. Early Stopping theo dõi loss trên tập validation qua tham số ‘patience’; nếu sau số epoch này mà validation loss không cải thiện, quá trình train bị dừng lại và mô hình khôi phục bộ trọng số có validation loss thấp nhất.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §2.3.

Bẫy đề thi cần tránh: Chờ Train loss về 0 (đó là dấu hiệu của Overfitting cực nặng).

Câu 41 (Đề thi thử VOA1 2026)

Đề bài: Điểm khác biệt cốt lõi giữa phương pháp Bagging (như Random Forest) và Boosting (như XGBoost) là:

- A. Bagging xây dựng các cây độc lập song song; Boosting xây dựng các cây tuần tự nối tiếp nhau.
- B. Bagging luôn cho độ chính xác cao hơn Boosting trên mọi tập dữ liệu.
- C. Boosting cố gắng giảm phương sai (Variance), Bagging cố gắng giảm độ chệch (Bias).
- D. Bagging chỉ dùng cho phân loại, Boosting chỉ dùng cho hồi quy.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: A**Phân tích & Chứng minh chi tiết:**

Bagging huấn luyện các cây độc lập song song trên các tập con bootstrap để giảm phương sai (Variance Reduction). Ngược lại, Boosting huấn luyện các cây tuần tự (Sequential): cây sau tập trung học để sửa sai số/phần dư (residuals) của các cây trước, nhằm giảm độ chệch (Bias Reduction).

Căn cứ tra cứu trong Handbook: Tham chiếu mục §1.4.

Bẫy đề thi cần tránh: Đảo ngược mục tiêu: Bagging giảm Variance, Boosting giảm Bias.

Câu 42 (Đề thi thử VOA1 2026)

Đề bài: Thuật toán CatBoost nổi tiếng trong cộng đồng Kaggle nhờ khả năng xử lý tự động và tối ưu loại dữ liệu nào mà không cần qua bước One-Hot Encoding?

- A. Dữ liệu dạng chuỗi thời gian (Time-series).
- B. Dữ liệu dạng phân loại (Categorical features).
- C. Dữ liệu ảnh màu RGB.
- D. Dữ liệu dạng âm thanh.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: B**Phân tích & Chứng minh chi tiết:**

CatBoost (Categorical Boosting) do Yandex phát triển, nổi tiếng với khả năng tự động xử lý các đặc trưng phân loại (Categorical features) có số lượng giá trị phân biệt lớn (High-cardinality) bằng kỹ thuật Target Statistics theo thứ tự ngẫu nhiên (Ordered Target Encoding), giải quyết triệt để vấn đề bùng nổ chiều của One-Hot Encoding và chống Target Leakage.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §1.4 & §7.4.

Bẫy đề thi cần tránh: Nhầm CatBoost là chuyên về Time-series (chữ ‘Cat’ là viết tắt của Categorical).

Câu 43 (Đề thi thử VOA1 2026)

Đề bài: Thuật toán LightGBM sử dụng chiến lược phát triển cây nào giúp nó thường chạy nhanh hơn và tốn ít bộ nhớ hơn so với XGBoost truyền thống?

- A. Leaf-wise growth (Phát triển theo chiều sâu của lá).
- B. Level-wise growth (Phát triển theo chiều ngang của tầng).
- C. Random growth (Phát triển ngẫu nhiên).
- D. Full-depth growth (Phát triển toàn bộ độ sâu cùng lúc).

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: A

Phân tích & Chứng minh chi tiết:

Hầu hết các thuật toán GBDT truyền thống (như XGBoost ban đầu) dùng Level-wise (phát triển cây cân bằng theo từng tầng ngang). LightGBM sử dụng chiến lược Leaf-wise (Best-first): tại mỗi bước, nó chỉ tìm chiếc lá có độ giảm hàm tổn thất (loss reduction) lớn nhất để phân nhánh, kết hợp phân giỏ Histogram giúp tốc độ huấn luyện nhanh gấp nhiều lần và tiết kiệm RAM.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §1.4.

Bẫy đề thi cần tránh: Level-wise là của XGBoost truyền thống, Leaf-wise mới là của LightGBM.

Câu 44 (Đề thi thử VOA1 2026)

Đề bài: Phương pháp 'Stacking' trong Ensemble learning hoạt động theo cơ chế nào?

- A. Lấy trung bình cộng đơn giản kết quả dự báo của tất cả các mô hình cơ sở.
- B. Dùng đầu ra (dự báo) của các mô hình cơ sở làm đầu vào cho một mô hình meta-learner cấp cao hơn.
- C. Nhân kết quả dự báo của các mô hình với một trọng số cố định được gán từ trước.
- D. Loại bỏ các mô hình yếu và chỉ giữ lại mô hình mạnh nhất.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: B

Phân tích & Chứng minh chi tiết:

Stacking (Stacked Generalization) huấn luyện nhiều mô hình cơ sở khác loại (Base learners: SVM, Random Forest, LightGBM...). Sau đó, các dự đoán Out-of-fold (OOF) của các mô hình này được ghép lại thành một ma trận đặc trưng mới để huấn luyện một mô hình cấp hai (Meta-learner, ví dụ Logistic Regression) nhằm học cách kết hợp tối ưu nhất.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §1.4.

Bẫy đề thi cần tránh: Nhầm Stacking với Voting/Averaging (lấy trung bình cộng đơn giản).

Câu 45 (Đề thi thử VOA1 2026)

Đề bài: Khi xây dựng một cây quyết định đơn lẻ trong thuật toán Random Forest, tại mỗi nút chia, thuật toán sẽ:

- A. Duyệt qua toàn bộ tất cả các đặc trưng hiện có để tìm điểm chia tốt nhất.
- B. Chỉ chọn ngẫu nhiên một tập con các đặc trưng để tìm điểm chia tốt nhất.
- C. Luôn chọn đặc trưng có giá trị trung bình lớn nhất.
- D. Không sử dụng bất kỳ tiêu chí ngẫu nhiên nào.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: B**Phân tích & Chứng minh chi tiết:**

Để triệt tiêu tương quan giữa các cây (Decorrelation of trees), tại mỗi nút chia, Random Forest chỉ lấy ngẫu nhiên một tập con đặc trưng (kích thước $m \approx \sqrt{p}$ cho phân loại, $m \approx p/3$ cho hồi quy) trong tổng số p đặc trưng để tìm điểm chia tối ưu. Nếu duyệt toàn bộ đặc trưng thì các cây sẽ có nhánh đầu giống hệt nhau (do đặc trưng mạnh nhất áp đảo), làm mất lợi thế giảm phương sai.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §1.4.

Bẫy đề thi cần tránh: Nghĩ rằng Random Forest duyệt qua toàn bộ p đặc trưng tại mỗi nút.

Câu 46 (Đề thi thử VOA1 2026)

Đề bài: (Khó) Mô hình FT-Transformer (Feature Tokenizer + Transformer) dành cho dữ liệu bảng (Tabular data) áp dụng cơ chế Self-Attention vào đâu?

- A. Giữa các hàng (các mẫu dữ liệu) khác nhau để tìm sự tương đồng.
- B. Giữa các đặc trưng (các cột) khác nhau sau khi đã được 'mã hóa' thành các token.
- C. Giữa các cây quyết định trong một rừng ngẫu nhiên.
- D. Nó không sử dụng Self-Attention mà dùng tích chập 1D.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: B**Phân tích & Chứng minh chi tiết:**

FT-Transformer (Gorishniy et al., NeurIPS 2021) biến đổi từng đặc trưng (cả cột số và cột phân loại) thành một vector nhúng (Feature Tokenizer). Sau đó, nó áp dụng chồng các khối Self-Attention giữa các vector đặc trưng này (Attention across features/columns trong cùng một mẫu), cho phép mô hình học các tương tác phi tuyến bậc cao giữa các thuộc tính.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §4.5 & §7.4.

Bẫy đề thi cần tránh: Nhầm lẫn giữa Attention theo hàng (Row/Sample attention) và Attention theo cột (Feature attention).

Câu 47 (Đề thi thử VOA1 2026)

Đề bài: Mô hình Neural Tabular nào gần đây nổi tiếng với việc chia sẻ tham số hiệu quả để tạo ra một 'ensemble' gồm nhiều mạng nơ-ron nhỏ bên trong một kiến trúc duy nhất?

- A. TabM.
- B. TabNet.
- C. NODE (Neural Oblivious Decision Ensembles).
- D. SAINT.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: A**Phân tích & Chứng minh chi tiết:**

TabM (Tabular Model with Mini-Ensembles, Gorishniy et al., ICLR 2025) giới thiệu kiến trúc mạng nơ-ron cho dữ liệu bảng chia sẻ phần lớn các tầng biểu diễn chung nhưng phân nhánh thành nhiều 'mini-ensembles' nhẹ bên trong, mang lại độ chính xác ngang ngửa hoặc vượt trội GBDT (XGBoost/CatBoost) mà tốc độ suy luận nhanh hơn nhiều so với việc ensemble nhiều model độc lập.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §1.4 & §7.4.

Bẫy đề thi cần tránh: Nhầm TabM với TabNet (TabNet dùng Sequential Attention Masking).

Câu 48 (Đề thi thử VOA1 2026)

Đề bài: Thuật toán Gradient Boosting xây dựng các cây quyết định dựa trên nguyên lý chủ đạo nào?

- A. Mỗi cây sau cố gắng học cách dự báo phần dư (residuals/errors) của các cây đứng trước nó.
- B. Các cây hoàn toàn độc lập và kết quả cuối cùng được bầu chọn theo số đông.
- C. Cây sau phải có độ sâu gấp đôi cây trước.
- D. Loại bỏ ngẫu nhiên 50% dữ liệu trước khi xây dựng cây mới.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: A

Phân tích & Chứng minh chi tiết:

Gradient Boosting xem việc huấn luyện là giải bài toán Gradient Descent trong không gian hàm. Mỗi cây mới $h_m(x)$ được khớp vào phần dư âm (negative gradient) của hàm loss tại dự đoán của mô hình hiện tại: $r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{m-1}}$. Với hàm loss MSE, giá trị này chính là sai số phần dư $y_i - F_{m-1}(x_i)$.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §1.4.

Bẫy đề thi cần tránh: Nhầm cơ chế học phần dư của Boosting với cơ chế độc lập của Bagging.

Câu 49 (Đề thi thử VOA1 2026)

Đề bài: Trong CatBoost, kỹ thuật 'Ordered Boosting' được thiết kế đặc biệt nhằm giải quyết triệt để vấn đề gì thường gặp ở các thuật toán Boosting khác?

- A. Hiện tượng Target leakage và Overfitting trong quá trình tính toán gradient trên tập dữ liệu nhỏ.
- B. Thời gian huấn luyện quá lâu trên CPU.
- C. Không thể chạy được trên GPU.
- D. Mô hình quá đơn giản dẫn đến Underfitting.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: A

Phân tích & Chứng minh chi tiết:

Trong GBDT truyền thống, gradient của mẫu thứ i được ước lượng bằng mô hình đã được xây dựng từ tập dữ liệu có chứa chính mẫu thứ i đó, gây ra độ lệch dự đoán (Prediction Shift / Target Leakage). Ordered Boosting xáo trộn ngẫu nhiên thứ tự tập dữ liệu và chỉ tính gradient cho mẫu thứ i dựa trên mô hình học từ các mẫu đứng trước nó theo thứ tự hoán vị, triệt tiêu rò rỉ thông tin.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §1.4.

Bẫy đề thi cần tránh: Nghĩ rằng Ordered Boosting chỉ là một kỹ thuật tối ưu tốc độ CPU/GPU.

Câu 50 (Đề thi thử VOA1 2026)

Đề bài: Tại sao việc áp dụng K-Fold cross-validation thông thường trực tiếp lên dữ liệu chuỗi thời gian (Time-series) để tuning mô hình lại bị coi là sai lầm nghiêm trọng?

- A. Vì nó phá vỡ tính tuần tự thời gian và dẫn đến rò rỉ dữ liệu từ tương lai vào quá khứ.
- B. Vì dữ liệu chuỗi thời gian không thể chia thành các Fold.
- C. Vì nó làm tăng gấp đôi kích thước của tập dữ liệu.
- D. Vì các mô hình chuỗi thời gian không có siêu tham số để tune.

[ĐÁP ÁN & LỜI GIẢI CHI TIẾT] — Đáp án đúng: A**Phân tích & Chứng minh chi tiết:**

Dữ liệu chuỗi thời gian có mối phụ thuộc thời gian (Temporal Dependence). K-Fold ngẫu nhiên sẽ xáo trộn các mốc thời gian, khiến mô hình lấy dữ liệu của ngày hôm sau để dự đoán cho ngày hôm trước (Look-ahead bias / Data leakage). Bắt buộc phải dùng phương pháp Time-Series Split (Rolling/Expanding Window) trong đó tập Train luôn diễn ra trước tập Validation.

Căn cứ tra cứu trong Handbook: Tham chiếu mục §1.5 & §5.6.

Bẫy đề thi cần tránh: Áp dụng rập khuôn K-Fold ngẫu nhiên cho bài toán chuỗi thời gian.

§9 Ma Trận Đối Chiếu Tra Cứu Web App & Lộ Trình Ôn Luyện Nước Rút

§9.1 Phương Pháp Kết Hợp Web App Thi Thử & Handbook PDF

Web App Ôn Thi OLP AI HCMUS 2026 (<https://lenamkhanhh.github.io/OLPAI26-Khanhdz/>) được thiết kế tối ưu cho việc rèn luyện phản xạ làm bài thi kiểu app GPLX 600 câu. Để đạt hiệu quả tối đa:

- Pha 1 — Luyện tập có phản hồi (Practice Mode trên Web):** Khi làm từng câu hỏi trong chế độ Practice, nếu trả lời sai hoặc chưa chắc chắn, đọc phần giải thích inline ngay dưới câu hỏi, đồng thời ghi chú mã mục lý thuyết (ví dụ §1.4, §3.1, §5.2).
- Pha 2 — Đọc sâu bản chất trong Handbook PDF:** Mở tài liệu này tại đúng mục § tương ứng để nắm vững công thức toán học, trực giác hình học, các tham số và các biến thể bẫy đề thi.
- Pha 3 — Thi thử tính giờ (Exam Mode trên Web):** Làm trọn vẹn đề thi 60 câu trắc nghiệm/code trong 90 phút mà không xem tài liệu. Đạt từ 70% điểm trở lên (ĐẠT) để đảm bảo năng lực vào đội tuyển.

§9.2 Bảng Ma Trận Ánh Xạ Chủ Đề Giữa Web App Và Handbook PDF

Module	Chủ đề / Kỹ năng trên Web App	Mục Tra Cứu Trong PDF
Module A (Toán & Xác suất)	Ma trận trực giao, nghịch đảo, định thức, chuẩn vector Cực trị hàm nhiều biến, Gradient, Ma trận Hessian Đạo hàm hàm kích hoạt, Softplus, Sigmoid Định lý Bayes, Xác suất hậu nghiệm, Base-rate fallacy Kỳ vọng, Phương sai, Quy tắc biến đổi tuyến tính MLE, MAP, Ước lượng hợp lý cực đại, Prior Gaussian Kiểm định giả thuyết, p-value, Ý nghĩa thống kê	§5.1 (Đại số tuyến tính nâng cao) §5.2 (Tối ưu hóa & Hessian) §2.2, §5.2 (Đạo hàm phi tuyến) §5.1 (Định lý Bayes) §5.3 (Kỳ vọng & Phương sai) §5.4 (MLE vs MAP) §5.5 (Kiểm định giả thuyết)
Module B (Python / Tính tay)	NumPy Vectorization, Broadcasting, Slicing mảng Tính toán kích thước đầu ra Conv2D: $(W - K + 2P)/S + 1$ Tính toán số lượng tham số lớp Conv2D Tính toán chỉ số IoU giữa hai bounding box Tính toán giá trị đầu ra nơ-ron: $\text{ReLU}(\mathbf{w}^T \mathbf{x} + b)$ Tính toán Precision, Recall, F1-score từ ma trận nhầm lẫn	§1.1, §1.9 (NumPy mảng đa chiều) §3.1 (Công thức tầng Conv2D) §3.1, Câu 14 VOAI §3.6 (Object Detection IoU) §2.1, Câu 20 VOAI §1.7, Câu 12 VOAI
Module C (ML / DL / CV / NLP)	k-NN (Lazy learner, Curse of dimensionality) SVM (Margin $2/\ \mathbf{w}\ $, Support vectors, RBF Kernel) Cây quyết định, Entropy $\log_2$, Information Gain Random Forest (Bagging), GBDT/XGBoost/LightGBM/CatBoost L1 (Lasso - Sparsity) vs L2 (Ridge - Co cụm) Imbalanced Data, SMOTE, Stratified K-Fold BatchNorm (Linear $\rightarrow$ Norm $\rightarrow$ ReLU) vs LayerNorm Double-Softmax Trap trong CrossEntropyLoss ResNet (Skip ADD) vs U-Net (Skip CONCAT) Vision Transformer (ViT, Patch embedding, không conv) Object Detection: NMS, YOLO Single-stage vs R-CNN NLP Pipeline, BoW, TF-IDF, Word2Vec, FastText, BERT Self-Attention Transformer: $\text{softmax}(QK^T/\sqrt{d_k})V$ RAG (Retrieval-Augmented Generation), Hallucination	§1.1 (k-NN) §1.2 (SVM) §1.3 (Decision Tree) §1.4 (Ensemble Learning) §1.6 (Regularization) §1.9, Câu 33, 35, 36 VOAI §2.8, Bẫy số 1 §2.4, Bẫy số 2 §3.4, Bẫy số 9 §3.5 (ViT) §3.6 (Detection) §4.1 – §4.6 (NLP) §4.5 (Transformer) §4.6, Câu 23 VOAI
Tự luận (C)	Khung giải pháp 5 bước chuẩn OLP AI cho 4 bài toán thực chiến	§7.1 – §7.4 (Chuyên đề thực chiến)

§9.3 Rubric Tự Chấm Điểm 4 Bài Tự Luận OLP AI (10 Điểm/Câu)

- Khối 1: Phân tích bài toán & Mục tiêu (2.0 điểm):** Xác định chuẩn xác dạng bài (Classification, Object Detection, Seq2Seq, Tabular Regression); phân tích đặc thù dữ liệu (mất cân bằng, độ phân giải, độ trễ thời gian thực) và ràng

buộc tài nguyên.

2. **Khối 2: Xử lý dữ liệu & Validation Strategy (2.0 điểm):** Đề xuất quy trình tiền xử lý phù hợp (chuẩn hóa, tokenization, data augmentation không rò rỉ); phương pháp chia tập nghiêm ngặt (Stratified K-Fold hoặc Time-Series Split, chống Data Leakage).
3. **Khối 3: Lựa chọn mô hình & Kiến trúc (2.5 điểm):** Đưa ra mô hình cơ sở (Baseline) → Đề xuất kiến trúc chính (YOLO, ResNet, U-Net, BERT, LightGBM/CatBoost); giải thích rõ tại sao kiến trúc này tối ưu cho bài toán.
4. **Khối 4: Chiến lược Huấn luyện & Tối ưu (2.0 điểm):** Lựa chọn Loss function chuyên biệt (Focal Loss, Cross-Entropy, CTC Loss); Optimizer (AdamW + Cosine Decay Warmup); kỹ thuật kiểm soát Overfitting (Dropout, Weight Decay, Early Stopping).
5. **Khối 5: Đánh giá, Phân tích Lỗi & Triển khai (1.5 điểm):** Sử dụng đúng metric chuẩn (mAP@0.5:0.95, F1-macro, BLEU, PR-AUC); phương án Error Analysis; kỹ thuật nén mô hình (Quantization INT8, ONNX Runtime, TensorRT) khi triển khai thực tế.

— HẾT CẨM NANG ÔN TẬP TOÀN DIỆN OLYMPIC AI HCMUS 2026 —

Chúc các thí sinh ôn tập vững vàng, làm chủ bản chất toán học và đạt kết quả xuất sắc trong kỳ thi Vòng loại 11/10/2026!